

Voice TicTacToe

Generated by Doxygen 1.9.8

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 defaults Namespace Reference	9
5.1.1 Variable Documentation	9
5.1.1.1 DEFAULT_BOARD_SIZE	9
5.1.1.2 DEFAULT_COLOR	9
5.1.1.3 DEFAULT_MINIMAX_DEPTH	9
5.1.1.4 GUI_CELL_COLUMN_SPAN	10
5.1.1.5 GUI_CELL_ROW_SPAN	10
5.1.1.6 INVALID_CELL	10
5.1.1.7 O_COLOR	10
5.1.1.8 X_COLOR	10
6 Class Documentation	11
6.1 AI Agent Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Constructor & Destructor Documentation	12
6.1.2.1 AI Agent()	12
6.1.2.2 ~AI Agent()	12
6.2 Board Class Reference	13
6.2.1 Detailed Description	13
6.2.2 Constructor & Destructor Documentation	14
6.2.2.1 Board() [1/4]	14
6.2.2.2 Board() [2/4]	14
6.2.2.3 Board() [3/4]	14
6.2.2.4 Board() [4/4]	14
6.2.2.5 ~Board()	15
6.2.3 Member Function Documentation	15
6.2.3.1 at()	15
6.2.3.2 evaluateBoard()	15
6.2.3.3 operator=()	16
6.2.3.4 printBoard()	16
6.2.3.5 reset()	16

6.2.3.6 resetCell()	16
6.2.3.7 setPlayerInput()	16
6.2.3.8 size()	17
6.2.4 Member Data Documentation	17
6.2.4.1 board_	17
6.2.4.2 boardSize_	18
6.3 Cell Struct Reference	18
6.3.1 Detailed Description	18
6.3.2 Constructor & Destructor Documentation	18
6.3.2.1 Cell()	18
6.3.3 Member Data Documentation	19
6.3.3.1 cellBtn	19
6.3.3.2 col	19
6.3.3.3 row	19
6.4 ConfigManager Class Reference	19
6.4.1 Detailed Description	20
6.4.2 Constructor & Destructor Documentation	20
6.4.2.1 ConfigManager() [1/2]	20
6.4.2.2 ~ConfigManager()	21
6.4.2.3 ConfigManager() [2/2]	21
6.4.3 Member Function Documentation	21
6.4.3.1 getBool()	21
6.4.3.2 getInt()	21
6.4.3.3 getValue()	22
6.4.3.4 instance()	22
6.4.3.5 load()	23
6.4.3.6 operator=()	23
6.4.3.7 resolvePath()	23
6.4.4 Member Data Documentation	23
6.4.4.1 baseDir_	23
6.4.4.2 settings_	24
6.5 HumanPlayer Class Reference	24
6.5.1 Detailed Description	25
6.5.2 Constructor & Destructor Documentation	25
6.5.2.1 HumanPlayer()	25
6.5.2.2 ~HumanPlayer()	26
6.5.3 Member Function Documentation	26
6.5.3.1 handleThreadCleanup()	26
6.5.3.2 parseSpeechCommand()	26
6.5.3.3 play()	27
6.5.3.4 startListening	27
6.5.3.5 stopListening	27

6.5.3.6 validateCommand()	27
6.5.4 Member Data Documentation	28
6.5.4.1 stt_	28
6.5.4.2 sttThread_	28
6.6 MiniMaxAgent Class Reference	28
6.6.1 Detailed Description	30
6.6.2 Constructor & Destructor Documentation	30
6.6.2.1 MiniMaxAgent()	30
6.6.3 Member Function Documentation	30
6.6.3.1 maxMove()	30
6.6.3.2 minMove()	31
6.6.3.3 play()	31
6.6.3.4 playerMark_()	32
6.6.3.5 score()	32
6.6.4 Member Data Documentation	32
6.6.4.1 AI_WIN_SCORE	32
6.6.4.2 depth_	33
6.6.4.3 PLAYER_WIN_SCORE	33
6.6.4.4 TIE_SCORE	33
6.7 NetworkManager Class Reference	33
6.7.1 Detailed Description	34
6.7.2 Constructor & Destructor Documentation	34
6.7.2.1 NetworkManager()	34
6.7.3 Member Function Documentation	35
6.7.3.1 connected	35
6.7.3.2 connectToServer()	35
6.7.3.3 disconnected	35
6.7.3.4 errorOccurred	35
6.7.3.5 messageReceived	36
6.7.3.6 messageSent	36
6.7.3.7 onConnected	36
6.7.3.8 onDisconnected	37
6.7.3.9 onError	37
6.7.3.10 onSocketReadyRead	37
6.7.3.11 sendMessage()	37
6.7.4 Member Data Documentation	38
6.7.4.1 socket_	38
6.8 Player Class Reference	38
6.8.1 Detailed Description	39
6.8.2 Constructor & Destructor Documentation	39
6.8.2.1 Player()	39
6.8.2.2 ~Player()	39

6.8.3 Member Function Documentation	39
6.8.3.1 play()	39
6.8.3.2 playerFinished	40
6.8.4 Member Data Documentation	40
6.8.4.1 board_	40
6.8.4.2 mark_	40
6.9 SpeechManager Class Reference	41
6.9.1 Detailed Description	42
6.9.2 Constructor & Destructor Documentation	42
6.9.2.1 SpeechManager()	42
6.9.2.2 ~SpeechManager()	42
6.9.3 Member Function Documentation	43
6.9.3.1 configure()	43
6.9.3.2 freeHeap()	43
6.9.3.3 hypIsValid()	43
6.9.3.4 init()	44
6.9.3.5 processRecognitionResult	44
6.9.3.6 speechReadyToProcess	44
6.9.3.7 speechRecognized	44
6.9.3.8 startListening()	45
6.9.4 Member Data Documentation	45
6.9.4.1 config_	45
6.9.4.2 decoder_	45
6.9.4.3 ep_	45
6.9.4.4 err_	46
6.9.4.5 frame_	46
6.9.4.6 frame_size_	46
6.9.4.7 hyp_	46
6.9.4.8 stream_	46
6.10 TicTacToeGame Class Reference	47
6.10.1 Detailed Description	48
6.10.2 Constructor & Destructor Documentation	48
6.10.2.1 TicTacToeGame()	48
6.10.3 Member Function Documentation	48
6.10.3.1 buildCellButtons()	48
6.10.3.2 declareGameState()	49
6.10.3.3 getBoardFinalStateText()	49
6.10.3.4 getPlayerStyleSheet()	49
6.10.3.5 getPlayerText()	50
6.10.3.6 goBack	50
6.10.3.7 newGame	50
6.10.3.8 reset()	50

6.10.3.9 setConnections()	51
6.10.3.10 setLedStyle()	51
6.10.3.11 updateCell()	51
6.10.3.12 updateConnectionStatus()	51
6.10.3.13 updateRecordingStatus()	52
6.10.3.14 updateWaitingStatus()	52
6.10.4 Member Data Documentation	52
6.10.4.1 ui	52
6.10.4.2 waiter	52
6.11 TitleScreen Class Reference	53
6.11.1 Detailed Description	54
6.11.2 Constructor & Destructor Documentation	54
6.11.2.1 TitleScreen()	54
6.11.3 Member Function Documentation	54
6.11.3.1 closeEvent()	54
6.11.3.2 exited	54
6.11.3.3 setConnections()	55
6.11.3.4 startGame	55
6.11.3.5 updateAlstartsGame	55
6.11.3.6 updateMiniMaxDepth	55
6.11.3.7 updateSinglePlayer	56
6.11.3.8 updateTwoPlayers	56
6.11.4 Member Data Documentation	56
6.11.4.1 options_	56
6.11.4.2 ui	56
6.12 TTTCController Class Reference	57
6.12.1 Detailed Description	58
6.12.2 Constructor & Destructor Documentation	58
6.12.2.1 TTTCController()	58
6.12.3 Member Function Documentation	58
6.12.3.1 getView()	58
6.12.3.2 onNetworkMessageReceived	59
6.12.3.3 playTurn	59
6.12.3.4 reset()	59
6.12.3.5 sendMessage	59
6.12.3.6 setConnections()	60
6.12.3.7 startGame()	60
6.12.3.8 switchPlayer()	60
6.12.3.9 updateGame	60
6.12.3.10 updateGameState()	61
6.12.4 Member Data Documentation	61
6.12.4.1 board_	61

6.12.4.2 cells_	61
6.12.4.3 currentPlayer_	61
6.12.4.4 network_	62
6.12.4.5 options_	62
6.12.4.6 playerO_	62
6.12.4.7 playerX_	62
6.12.4.8 view_	62
6.13 TTOptions Struct Reference	62
6.13.1 Detailed Description	63
6.13.2 Member Data Documentation	63
6.13.2.1 Alopponent	63
6.13.2.2 Alstarts	63
6.13.2.3 boardSize	63
6.13.2.4 miniMaxDepth	63
6.14 WaitingSpinnerWidget Class Reference	63
6.14.1 Constructor & Destructor Documentation	65
6.14.1.1 WaitingSpinnerWidget() [1/3]	65
6.14.1.2 WaitingSpinnerWidget() [2/3]	65
6.14.1.3 WaitingSpinnerWidget() [3/3]	66
6.14.2 Member Function Documentation	66
6.14.2.1 color()	66
6.14.2.2 currentLineColor()	66
6.14.2.3 initialize()	66
6.14.2.4 innerRadius()	66
6.14.2.5 isSpinning()	66
6.14.2.6 lineCountDistanceFromPrimary()	67
6.14.2.7 lineLength()	67
6.14.2.8 lineWidth()	67
6.14.2.9 minimumTrailOpacity()	67
6.14.2.10 numberOfLines()	67
6.14.2.11 operator=()	67
6.14.2.12 paintEvent()	67
6.14.2.13 revolutionsPersSecond()	68
6.14.2.14 rotate	68
6.14.2.15 roundness()	68
6.14.2.16 setColor()	68
6.14.2.17 setInnerRadius()	68
6.14.2.18 setLineLength()	68
6.14.2.19 setLineWidth()	68
6.14.2.20 setMinimumTrailOpacity()	69
6.14.2.21 setNumberOfLines()	69
6.14.2.22 setRevolutionsPerSecond()	69

6.14.2.23 setRoundness()	69
6.14.2.24 setText()	69
6.14.2.25 setTrailFadePercentage()	69
6.14.2.26 start	69
6.14.2.27 stop	70
6.14.2.28 trailFadePercentage()	70
6.14.2.29 updatePosition()	70
6.14.2.30 updateSize()	70
6.14.2.31 updateTimer()	70
6.14.3 Member Data Documentation	70
6.14.3.1 _centerOnParent	70
6.14.3.2 _color	71
6.14.3.3 _currentCounter	71
6.14.3.4 _disableParentWhenSpinning	71
6.14.3.5 _innerRadius	71
6.14.3.6 _isSpinning	71
6.14.3.7 _lineLength	71
6.14.3.8 _lineWidth	71
6.14.3.9 _minimumTrailOpacity	71
6.14.3.10 _numberOfLines	72
6.14.3.11 _revolutionsPerSecond	72
6.14.3.12 _roundness	72
6.14.3.13 _timer	72
6.14.3.14 _trailFadePercentage	72
7 File Documentation	73
7.1 docs/plantuml/AIAgent.puml File Reference	73
7.2 docs/plantuml/Board.puml File Reference	73
7.3 docs/plantuml/ConfigManager.puml File Reference	73
7.4 docs/plantuml/HumanPlayer.puml File Reference	73
7.5 docs/plantuml/MiniMaxAgent.puml File Reference	73
7.6 docs/plantuml/NetworkManager.puml File Reference	73
7.7 docs/plantuml/Player.puml File Reference	73
7.8 docs/plantuml/SpeechManager.puml File Reference	73
7.9 docs/plantuml/TicTacToeGame.puml File Reference	73
7.10 docs/plantuml/TitleScreen.puml File Reference	73
7.11 docs/plantuml/TTTCommonTypes.puml File Reference	73
7.12 docs/plantuml/TTTController.puml File Reference	73
7.13 docs/plantuml/WaitingSpinnerWidget.puml File Reference	73
7.14 include/AI/AIAgent.h File Reference	73
7.14.1 Detailed Description	74
7.15 AIAgent.h	74

7.16 include/AI/HumanPlayer.h File Reference	74
7.16.1 Detailed Description	75
7.17 HumanPlayer.h	75
7.18 include/AI/MiniMaxAgent.h File Reference	75
7.18.1 Detailed Description	76
7.19 MiniMaxAgent.h	76
7.20 include/AI/Player.h File Reference	77
7.20.1 Detailed Description	77
7.21 Player.h	77
7.22 include/Config/ConfigManager.h File Reference	78
7.22.1 Detailed Description	78
7.23 ConfigManager.h	78
7.24 include/Controller/TTTController.h File Reference	79
7.24.1 Detailed Description	79
7.25 TTTController.h	80
7.26 include/Model/Board.h File Reference	80
7.26.1 Detailed Description	81
7.27 Board.h	81
7.28 include/Network/NetworkManager.h File Reference	82
7.28.1 Detailed Description	82
7.29 NetworkManager.h	82
7.30 include/Speech/SpeechManager.h File Reference	83
7.30.1 Detailed Description	83
7.31 SpeechManager.h	84
7.32 include/TTTCommonTypes.h File Reference	84
7.32.1 Detailed Description	85
7.32.2 Enumeration Type Documentation	85
7.32.2.1 BoardMarks	85
7.32.2.2 BoardState	86
7.33 TTTCommonTypes.h	86
7.34 include/View/TicTacToeGame.h File Reference	87
7.34.1 Detailed Description	87
7.35 TicTacToeGame.h	87
7.36 include/View/TitleScreen.h File Reference	88
7.36.1 Detailed Description	88
7.37 TitleScreen.h	89
7.38 include/View/waitingspinnerwidget.h File Reference	89
7.39 waitingspinnerwidget.h	89
7.40 src/AI/HumanPlayer.cpp File Reference	91
7.41 src/AI/MiniMaxAgent.cpp File Reference	91
7.42 src/AI/Player.cpp File Reference	91
7.43 src/Config/ConfigManager.cpp File Reference	91

7.44 src/Controller/TTTController.cpp File Reference	91
7.45 src/main.cpp File Reference	91
7.45.1 Function Documentation	92
7.45.1.1 main()	92
7.46 src/Model/Board.cpp File Reference	92
7.47 src/Network/NetworkManager.cpp File Reference	92
7.48 src/Speech/SpeechManager.cpp File Reference	92
7.49 src/View/TicTacToeGame.cpp File Reference	92
7.50 src/View/TitleScreen.cpp File Reference	92
7.51 src/waitingspinnerwidget.cpp File Reference	93
Index	95

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

defaults		9
--------------------------	-------	---

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Board	13
Cell	18
QDialog	
TicTacToeGame	47
QMainWindow	
TitleScreen	53
QObject	
ConfigManager	19
NetworkManager	33
Player	38
AIAgent	11
MiniMaxAgent	28
HumanPlayer	24
SpeechManager	41
TTTController	57
QWidget	
WaitingSpinnerWidget	63
TTTOptions	62

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AIAgent	The AIAgent interface describes the behavior of any implemented Artificial Intelligence agent . . .	11
Board	Responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board	13
Cell	The Cell struct represents a board cell and all the necessary information needed about the cell	18
ConfigManager	Manages the application configuration	19
HumanPlayer	Human player in the game	24
MiniMaxAgent	Implements the logic to select the best move using the Minimax algorithm	28
NetworkManager	TCP communication manager class	33
Player	Abstract base class for different types of players	38
SpeechManager	Voice recognition manager class using Pocketsphinx and PortAudio	41
TicTacToeGame	Responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one	47
TitleScreen	Responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options	53
TTTController	Acts as the controller (C in MVC) for a Tic-Tac-Toe game	57
TTTOptions	The TTTOptions struct defines a tic tac toe game options	62
WaitingSpinnerWidget		63

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

docs/plantuml/ AIAgent.puml	73
docs/plantuml/ Board.puml	73
docs/plantuml/ ConfigManager.puml	73
docs/plantuml/ HumanPlayer.puml	73
docs/plantuml/ MiniMaxAgent.puml	73
docs/plantuml/ NetworkManager.puml	73
docs/plantuml/ Player.puml	73
docs/plantuml/ SpeechManager.puml	73
docs/plantuml/ TicTacToeGame.puml	73
docs/plantuml/ TitleScreen.puml	73
docs/plantuml/ TTTCommonTypes.puml	73
docs/plantuml/ TTTController.puml	73
docs/plantuml/ WaitingSpinnerWidget.puml	73
include/ TTTCommonTypes.h Definition of common types and constants for the Tic-Tac-Toe game	84
include/AI/ AIAgent.h Definition of the AIAgent interface for artificial intelligence agents	73
include/AI/ HumanPlayer.h Definition of the HumanPlayer class that represents the human player	74
include/AI/ MiniMaxAgent.h Definition of the MiniMaxAgent class implementing the Minimax algorithm with Alpha-Beta pruning	75
include/AI/ Player.h Definition of the Player abstract base class for different player types	77
include/Config/ ConfigManager.h Definition of the ConfigManager class for application configuration management	78
include/Controller/ TTTController.h Definition of the TTTController class for the Tic-Tac-Toe game controller (MVC pattern)	79
include/Model/ Board.h Definition of the Board class representing the game model	80
include/Network/ NetworkManager.h Definition of the NetworkManager class for TCP communication	82
include/Speech/ SpeechManager.h Definition of the SpeechManager class for voice recognition using Pocketsphinx and PortAudio	83
include/View/ TicTacToeGame.h Definition of the TicTacToeGame class for the game GUI	87

include/View/TitleScreen.h	
Definition of the TitleScreen class for the main menu GUI	88
include/View/ waitingspinnerwidget.h	89
src/ main.cpp	91
src/ waitingspinnerwidget.cpp	93
src/Al/ HumanPlayer.cpp	91
src/Al/ MiniMaxAgent.cpp	91
src/Al/ Player.cpp	91
src/Config/ ConfigManager.cpp	91
src/Controller/ TTTController.cpp	91
src/Model/ Board.cpp	92
src/Network/ NetworkManager.cpp	92
src/Speech/ SpeechManager.cpp	92
src/View/ TicTacToeGame.cpp	92
src/View/ TitleScreen.cpp	92

Chapter 5

Namespace Documentation

5.1 defaults Namespace Reference

Variables

- constexpr short [INVALID_CELL](#) = -1
- constexpr size_t [DEFAULT_BOARD_SIZE](#) = 3
- constexpr unsigned short [DEFAULT_MINIMAX_DEPTH](#) = 3
- constexpr unsigned short [GUI_CELL_ROW_SPAN](#) = 1
- constexpr unsigned short [GUI_CELL_COLUMN_SPAN](#) = 1
- constexpr char [X_COLOR](#) [] = "#455A64"
- constexpr char [O_COLOR](#) [] = "#FF5722"
- constexpr char [DEFAULT_COLOR](#) [] = "#FFF"

5.1.1 Variable Documentation

5.1.1.1 DEFAULT_BOARD_SIZE

```
constexpr size_t defaults::DEFAULT_BOARD_SIZE = 3 [constexpr]
```

5.1.1.2 DEFAULT_COLOR

```
constexpr char defaults::DEFAULT_COLOR[] = "#FFF" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

5.1.1.3 DEFAULT_MINIMAX_DEPTH

```
constexpr unsigned short defaults::DEFAULT_MINIMAX_DEPTH = 3 [constexpr]
```

5.1.1.4 GUI_CELL_COLUMN_SPAN

```
constexpr unsigned short defaults::GUI_CELL_COLUMN_SPAN = 1 [constexpr]
```

Referenced by [TicTacToeGame::buildCellButtons\(\)](#).

5.1.1.5 GUI_CELL_ROW_SPAN

```
constexpr unsigned short defaults::GUI_CELL_ROW_SPAN = 1 [constexpr]
```

Referenced by [TicTacToeGame::buildCellButtons\(\)](#).

5.1.1.6 INVALID_CELL

```
constexpr short defaults::INVALID_CELL = -1 [constexpr]
```

5.1.1.7 O_COLOR

```
constexpr char defaults::O_COLOR[] = "#FF5722" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

5.1.1.8 X_COLOR

```
constexpr char defaults::X_COLOR[] = "#455A64" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

Chapter 6

Class Documentation

6.1 AIAgent Class Reference

The [AIAgent](#) interface describes the behavior of any implemented Artificial Intelligence agent.

```
#include <AIAgent.h>
```

Inherits [Player](#).

Inherited by [MiniMaxAgent](#).

Public Member Functions

- [AIAgent](#) ([BoardMarks](#) playerMark, const [Board](#) &board_ref, QObject *parent_=nullptr)
Constructor for the [AIAgent](#) base class.
- virtual [~AIAgent](#) ()
Virtual destructor.

Public Member Functions inherited from [Player](#)

- [Player](#) (const [BoardMarks](#) &playerMark, const [Board](#) &board_ref, QObject *parent_=nullptr)
Constructor for the base [Player](#) class.
- virtual [~Player](#) ()
Virtual destructor.
- virtual void [play](#) ()=0
Pure abstract method to start the player's move logic.

Additional Inherited Members

Signals inherited from [Player](#)

- void [playerFinished](#) (int const &cell)
Signal emitted when the player has completed their move.

Protected Attributes inherited from [Player](#)

- const [Board](#) * [board_](#)
Constant pointer to the game board.
- const [BoardMarks](#) & [mark_](#)
Constant reference to the marker assigned to this player (X or O).

6.1.1 Detailed Description

The [AIAgent](#) interface describes the behavior of any implemented Artificial Intelligence agent.

This abstract base class inherits from [Player](#) and establishes a common interface for all AI agents (such as [MiniMaxAgent](#)). It defines the basic functions necessary for an agent to reset its internal state before a new turn or game.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 AIAgent()

```
AIAgent::AIAgent (
    BoardMarks playerMark,
    const Board & borad_ref,
    QObject * parent_ = nullptr ) [inline]
```

Constructor for the [AIAgent](#) base class.

Initializes the agent with its marker and a reference to the board.

Parameters

<i>playerMark</i>	The marker (BoardMarks::X or BoardMarks::O) assigned to the AI.
<i>borad_ref</i>	Constant reference to the game Board object (Model).
<i>parent_</i>	Pointer to the parent QObject (nullptr by default).

6.1.2.2 ~AIAgent()

```
virtual AIAgent::~AIAgent ( ) [inline], [virtual]
```

Virtual destructor.

Allows proper destruction of derived classes through a base class pointer.

The documentation for this class was generated from the following file:

- include/AI/[AIAgent.h](#)

6.2 Board Class Reference

The `Board` class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

```
#include <Board.h>
```

Public Member Functions

- `Board (size_t size)`
Constructs a board given a board size.
- `Board (Board &b)`
Board copy constructor.
- `Board (const Board &b)`
Board copy of const constructor.
- `Board (Board &&b)`
Board move constructor.
- `Board & operator= (Board b)`
This assignment operator implements the copy & swap idiom, so it works as a move and a copy assignment operator.
- `virtual ~Board ()`
Board destructor.
- `virtual BoardState evaluateBoard () const`
evaluateBoard evaluates whether the state of the current board. the function can evaluate to X wins or O wins or a Tie or No Winner.
- `virtual bool setPlayerInput (size_t row, size_t col, BoardMarks currentPlayer)`
setPlayerInput sets the board with the mark given in the specified location. after boundary checking the input row and column, and if the game hasn't reached a final state.
- `virtual BoardMarks at (size_t row, size_t col) const`
at is a getter function that returns a cell given a row and a column.
- `virtual void reset ()`
reset allows resetting the whole board, marking all the cells as empty.
- `virtual void resetCell (size_t row, size_t col)`
resetCell allows resetting a given cell to an empty type, without the extra state checks done in the setPlayerInput function.
- `size_t size () const`
size is a getter functions of the board size.
- `virtual void printBoard () const`

Protected Attributes

- `vector< vector< BoardMarks > > board_`
board is a 2D vector that represents a machine readable format to the GUI application.
- `size_t boardSize_`
boardSize is the number of rows or columns in the board. a 3x3 board has a boardSize of 3.

6.2.1 Detailed Description

The `Board` class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Board() [1/4]

```
Board::Board (
    size_t size )
```

Constructs a board given a board size.

Parameters

<i>size</i>	is the number of rows or columns in the board.
-------------	--

6.2.2.2 Board() [2/4]

```
Board::Board (
    Board & b )
```

[Board](#) copy constructor.

Parameters

<i>b</i>	The board to be copied.
----------	-------------------------

6.2.2.3 Board() [3/4]

```
Board::Board (
    const Board & b )
```

[Board](#) copy of const constructor.

Parameters

<i>b</i>	The const board to be copied.
----------	-------------------------------

6.2.2.4 Board() [4/4]

```
Board::Board (
    Board && b )
```

[Board](#) move constructor.

Parameters

<i>b</i>	The board to be moved.
----------	------------------------

6.2.2.5 ~Board()

```
Board::~~Board ( ) [virtual]
```

[Board](#) destructor.

6.2.3 Member Function Documentation

6.2.3.1 at()

```
BoardMarks Board::at (
    size_t row,
    size_t col ) const [virtual]
```

at is a getter function that returns a cell given a row and a column.

Parameters

<i>row</i>	of the cell target.
<i>col</i>	of the cell target.

Returns

the value mark at the cell.

Exceptions

<i>out_of_range</i>	exception when the row or column is out of bounds.
---------------------	--

References [board_](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [HumanPlayer::parseSpeechCommand\(\)](#), and [MiniMaxAgent::play\(\)](#).

6.2.3.2 evaluateBoard()

```
BoardState Board::evaluateBoard ( ) const [virtual]
```

evaluateBoard evaluates whether the state of the current board. the function can evaluate to X wins or O wins or a Tie or No Winner.

Returns

the state of the current board.

References [board_](#), [boardSize_](#), [Empty](#), [NoWinner](#), [O](#), [OWins](#), [Tie](#), [X](#), and [XWins](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [TTTController::onNetworkMessageReceived\(\)](#), [MiniMaxAgent::play\(\)](#), [setPlayerInput\(\)](#), and [TTTController::updateGameState\(\)](#).

6.2.3.3 operator=()

```
Board & Board::operator= (
    Board b )
```

This assignment operator implements the copy & swap idiom, so it works as a move and a copy assignment operator.

Parameters

<i>b</i>	The board to be copied or moved.
----------	----------------------------------

Returns

a reference to this object, so this operator can be chained.

References [board_](#), and [boardSize_](#).

6.2.3.4 printBoard()

```
virtual void Board::printBoard ( ) const [virtual]
```

Referenced by [TTTController::updateGameState\(\)](#).

6.2.3.5 reset()

```
void Board::reset ( ) [virtual]
```

reset allows resetting the whole board, marking all the cells as empty.

References [board_](#), [boardSize_](#), and [Empty](#).

Referenced by [TTTController::reset\(\)](#).

6.2.3.6 resetCell()

```
void Board::resetCell (
    size_t row,
    size_t col ) [virtual]
```

resetCell allows resetting a given cell to an empty type, without the extra state checks done in the setPlayerInput function.

References [board_](#), [boardSize_](#), and [Empty](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), and [MiniMaxAgent::play\(\)](#).

6.2.3.7 setPlayerInput()

```
bool Board::setPlayerInput (
    size_t row,
    size_t col,
    BoardMarks currentPlayer ) [virtual]
```

setPlayerInput sets the board with the mark given in the specified location. after boundary checking the input row and column, and if the game hasn't reached a final state.

Parameters

<i>row</i>	of the cell target.
<i>col</i>	of the cell target.
<i>currentPlayer</i>	is the mark of the current player.

Returns

whether the update was successfull or not based on the boundary check and validity of input given the board state.

Exceptions

<i>out_of_range</i>	exception when the row or column is out of bounds.
---------------------	--

References [board_](#), [boardSize_](#), [Empty](#), [evaluateBoard\(\)](#), and [NoWinner](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [MiniMaxAgent::play\(\)](#), and [TTTController::updateGame\(\)](#).

6.2.3.8 size()

```
size_t Board::size ( ) const
```

size is a getter functions of the board size.

Returns

the size of the board i.e. numebr of rows or columns.

References [boardSize_](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [HumanPlayer::parseSpeechCommand\(\)](#), and [MiniMaxAgent::play\(\)](#).

6.2.4 Member Data Documentation**6.2.4.1 board_**

```
vector<vector<BoardMarks> > Board::board_ [protected]
```

board is a 2D vector that represents a machine readable format to the GUI application.

Referenced by [at\(\)](#), [evaluateBoard\(\)](#), [operator=\(\)](#), [reset\(\)](#), [resetCell\(\)](#), and [setPlayerInput\(\)](#).

6.2.4.2 boardSize_

```
size_t Board::boardSize_ [protected]
```

boardSize is the number of rows or columns in the board. a 3x3 board has a boardSize of 3.

Referenced by [evaluateBoard\(\)](#), [operator=\(\)](#), [reset\(\)](#), [resetCell\(\)](#), [setPlayerInput\(\)](#), and [size\(\)](#).

The documentation for this class was generated from the following files:

- [include/Model/Board.h](#)
- [src/Model/Board.cpp](#)

6.3 Cell Struct Reference

The [Cell](#) struct represents a board cell and all the necessary information needed about the cell.

```
#include <TTTCommonTypes.h>
```

Public Member Functions

- [Cell](#) (QPushButton *[cellBtn](#), int [row](#), int [col](#))

Public Attributes

- QPushButton * [cellBtn](#) = nullptr
- int [row](#) = [defaults::INVALID_CELL](#)
- int [col](#) = [defaults::INVALID_CELL](#)

6.3.1 Detailed Description

The [Cell](#) struct represents a board cell and all the necessary information needed about the cell.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Cell()

```
Cell::Cell (  
    QPushButton * cellBtn,  
    int row,  
    int col ) [inline]
```

6.3.3 Member Data Documentation

6.3.3.1 cellBtn

```
QPushButton* Cell::cellBtn = nullptr
```

Referenced by [TicTacToeGame::updateCell\(\)](#).

6.3.3.2 col

```
int Cell::col = defaults::INVALID_CELL
```

Referenced by [TTTController::updateGame\(\)](#).

6.3.3.3 row

```
int Cell::row = defaults::INVALID_CELL
```

Referenced by [TTTController::updateGame\(\)](#).

The documentation for this struct was generated from the following file:

- include/[TTTCommonTypes.h](#)

6.4 ConfigManager Class Reference

The [ConfigManager](#) class manages the application configuration.

```
#include <ConfigManager.h>
```

Inherits [QObject](#).

Public Member Functions

- void [load](#) (const QString &filePath=QString())
Loads the configuration file from the specified path.
- QString [getValue](#) (const QString &key, const QString &defaultValue=QString()) const
Returns a value from the INI file as QString.
- int [getInt](#) (const QString &key, int defaultValue=0) const
Returns a value from the INI file as integer.
- bool [getBool](#) (const QString &key, bool defaultValue=false) const
Returns a value from the INI file as boolean.
- QString [resolvePath](#) (const QString &relativePath) const
Resolves a relative path using the application base directory.

Static Public Member Functions

- static [ConfigManager](#) & [instance](#) ()
Gets the unique instance of [ConfigManager](#) (Singleton).

Private Member Functions

- [ConfigManager](#) (QObject *parent=nullptr)
Private constructor.
- [~ConfigManager](#) () override=default
Destructor.
- [ConfigManager](#) (const [ConfigManager](#) &)=delete
Copy constructor blocked.
- [ConfigManager](#) & [operator=](#) (const [ConfigManager](#) &)=delete
Assignment operator blocked.

Private Attributes

- QSettings * [settings_](#) = nullptr
Pointer to the QSettings object that manages the INI file.
- QString [baseDir_](#)
The base directory of the loaded configuration file.

6.4.1 Detailed Description

The [ConfigManager](#) class manages the application configuration.

Manages the application configuration from a config.ini file. Applies the **Singleton** pattern to ensure only one globally accessible instance exists, providing a centralized point to access program options.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 ConfigManager() [1/2]

```
ConfigManager::ConfigManager (
    QObject * parent = nullptr ) [explicit], [private]
```

Private constructor.

Private to ensure the Singleton pattern.

Parameters

<i>parent</i>	Pointer to the parent object (QObject).
---------------	---

6.4.2.2 ~ConfigManager()

```
ConfigManager::~ConfigManager ( ) [override], [private], [default]
```

Destructor.

Uses = default for standard implementation.

6.4.2.3 ConfigManager() [2/2]

```
ConfigManager::ConfigManager (
    const ConfigManager & ) [private], [delete]
```

Copy constructor blocked.

6.4.3 Member Function Documentation

6.4.3.1 getBool()

```
bool ConfigManager::getBool (
    const QString & key,
    bool defaultValue = false ) const
```

Returns a value from the INI file as boolean.

Converts the key value to a boolean (true if it's "true", "on", "1", etc.).

Parameters

<i>key</i>	Configuration key (e.g., "ai/enabled").
<i>defaultValue</i>	Default value if the key is not found.

Returns

Value associated with the key as bool.

References [settings_](#).

6.4.3.2 getInt()

```
int ConfigManager::getInt (
    const QString & key,
    int defaultValue = 0 ) const
```

Returns a value from the INI file as integer.

Converts the key value to an integer.

Parameters

<i>key</i>	Configuration key (e.g., "audio/samplerate").
<i>defaultValue</i>	Default value if the key is not found.

Returns

Value associated with the key as int.

References [settings_](#).

Referenced by [TTTController::setConnections\(\)](#).

6.4.3.3 getValue()

```
QString ConfigManager::getValue (
    const QString & key,
    const QString & defaultValue = QString() ) const
```

Returns a value from the INI file as QString.

Reads a value from the associated QSettings. If the key doesn't exist, returns the default value.

Parameters

<i>key</i>	Configuration key (e.g., "network/ip").
<i>defaultValue</i>	Default value if the key is not found.

Returns

Value associated with the key as QString.

References [settings_](#).

Referenced by [TTTController::setConnections\(\)](#).

6.4.3.4 instance()

```
ConfigManager & ConfigManager::instance ( ) [static]
```

Gets the unique instance of [ConfigManager](#) (Singleton).

This is the global access point to the configuration manager.

Returns

Reference to the unique [ConfigManager](#) object.

References [instance\(\)](#).

Referenced by [SpeechManager::configure\(\)](#), [instance\(\)](#), [main\(\)](#), and [TTTController::setConnections\(\)](#).

6.4.3.5 load()

```
void ConfigManager::load (
    const QString & filePath = QString() )
```

Loads the configuration file from the specified path.

If no path is specified, it will look for a `config.ini` file in the current application directory. Initializes the internal `QSettings` object.

Parameters

<i>filePath</i>	Complete path to the .ini file. If empty, uses the default value.
-----------------	---

References [baseDir_](#), and [settings_](#).

Referenced by [main\(\)](#).

6.4.3.6 operator=()

```
ConfigManager & ConfigManager::operator= (
    const ConfigManager & ) [private], [delete]
```

Assignment operator blocked.

6.4.3.7 resolvePath()

```
QString ConfigManager::resolvePath (
    const QString & relativePath ) const
```

Resolves a relative path using the application base directory.

Uses the base directory established during loading to convert a relative path (e.g., "data/model.dat") to an absolute path.

Parameters

<i>relativePath</i>	The relative path to resolve.
---------------------	-------------------------------

Returns

The calculated absolute path.

References [baseDir_](#).

6.4.4 Member Data Documentation

6.4.4.1 baseDir_

```
QString ConfigManager::baseDir_ [private]
```

The base directory of the loaded configuration file.

Referenced by [load\(\)](#), and [resolvePath\(\)](#).

6.4.4.2 settings_

```
QSettings* ConfigManager::settings_ = nullptr [private]
```

Pointer to the QSettings object that manages the INI file.

Referenced by [getBool\(\)](#), [getInt\(\)](#), [getValue\(\)](#), and [load\(\)](#).

The documentation for this class was generated from the following files:

- include/Config/[ConfigManager.h](#)
- src/Config/[ConfigManager.cpp](#)

6.5 HumanPlayer Class Reference

The [HumanPlayer](#) class represents the human player in the game.

```
#include <HumanPlayer.h>
```

Inherits [Player](#).

Signals

- void [startListening](#) ()
Signal emitted to start microphone listening in the worker thread.
- void [stopListening](#) ()
Signal emitted to stop microphone listening and shut down the worker thread.

Signals inherited from [Player](#)

- void [playerFinished](#) (int const &cell)
Signal emitted when the player has completed their move.

Public Member Functions

- [HumanPlayer](#) (const [Board](#) &board_ref, [BoardMarks](#) playerMark=[BoardMarks::Empty](#), QObject *parent=nullptr)
Constructor for the human player.
- virtual [~HumanPlayer](#) ()
Destructor for the human player.
- void [play](#) () override
Starts the human player's turn.

Public Member Functions inherited from [Player](#)

- [Player](#) (const [BoardMarks](#) &playerMark, const [Board](#) &board_ref, QObject *parent_=nullptr)
Constructor for the base [Player](#) class.
- virtual [~Player](#) ()
Virtual destructor.

Private Member Functions

- void [parseSpeechCommand](#) (const QString &text)
Processes the text recognized by the [SpeechManager](#).
- bool [validateCommand](#) (const QString &command)
Validates whether the recognized voice command corresponds to a valid move.
- void [handleThreadCleanup](#) ()
Slot responsible for managing the worker thread lifecycle.

Private Attributes

- [SpeechManager](#) * [stt_](#)
Pointer to the [SpeechManager](#) object, the worker for voice recognition.
- QThread * [sttThread_](#)
Pointer to the worker thread dedicated to running the [SpeechManager](#).

Additional Inherited Members

Protected Attributes inherited from [Player](#)

- const [Board](#) * [board_](#)
Constant pointer to the game board.
- const [BoardMarks](#) & [mark_](#)
Constant reference to the marker assigned to this player (X or O).

6.5.1 Detailed Description

The [HumanPlayer](#) class represents the human player in the game.

Inherits from the abstract base class [Player](#). It is responsible for capturing user input (either through GUI clicks or, in this case, through voice recognition using [SpeechManager](#)) and converting it into a valid move on the board.

Uses a dedicated worker thread (QThread) for voice processing ([SpeechManager](#)) to avoid blocking the main GUI thread.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 HumanPlayer()

```
HumanPlayer::HumanPlayer (
    const Board & board_ref,
    BoardMarks playerMark = BoardMarks::Empty,
    QObject * parent = nullptr ) [explicit]
```

Constructor for the human player.

Initializes the reference to the board and the player's marker. Also creates and initializes the [SpeechManager](#) object and its associated thread.

Parameters

<i>board_ref</i>	Constant reference to the board model.
<i>playerMark</i>	The marker (BoardMarks::X or BoardMarks::O) assigned to the player.
<i>parent</i>	Pointer to the parent QObject.

References [SpeechManager::configure\(\)](#), [parseSpeechCommand\(\)](#), [SpeechManager::speechRecognized\(\)](#), [startListening\(\)](#), [SpeechManager::startListening\(\)](#), [stt_](#), and [sttThread_](#).

6.5.2.2 ~HumanPlayer()

```
virtual HumanPlayer::~~HumanPlayer ( ) [inline], [virtual]
```

Destructor for the human player.

Handles the cleanup and safe termination of the voice recognition thread.

6.5.3 Member Function Documentation**6.5.3.1 handleThreadCleanup()**

```
void HumanPlayer::handleThreadCleanup ( ) [private]
```

Slot responsible for managing the worker thread lifecycle.

Executes the safe thread termination process (`quit()`, `wait()`) and resource cleanup (`QThread::deleteLater()`).

6.5.3.2 parseSpeechCommand()

```
void HumanPlayer::parseSpeechCommand (
    const QString & text ) [private]
```

Processes the text recognized by the [SpeechManager](#).

This method receives the text hypothesis (`text`) from the voice recognition system and initiates command validation.

Parameters

<i>text</i>	The recognized text string (voice command).
-------------	---

References [Board::at\(\)](#), [Player::board_](#), [Empty](#), [Player::playerFinished\(\)](#), [Board::size\(\)](#), [startListening\(\)](#), and [stopListening\(\)](#).

Referenced by [HumanPlayer\(\)](#).

6.5.3.3 play()

```
void HumanPlayer::play ( ) [override], [virtual]
```

Starts the human player's turn.

This overridden method starts the voice recognition thread ([SpeechManager](#)) and instructs it to begin listening, waiting for a user command.

Implements [Player](#).

References [startListening\(\)](#).

6.5.3.4 startListening

```
void HumanPlayer::startListening ( ) [signal]
```

Signal emitted to start microphone listening in the worker thread.

Referenced by [HumanPlayer\(\)](#), [parseSpeechCommand\(\)](#), [play\(\)](#), and [TTTController::TTTController\(\)](#).

6.5.3.5 stopListening

```
void HumanPlayer::stopListening ( ) [signal]
```

Signal emitted to stop microphone listening and shut down the worker thread.

Referenced by [parseSpeechCommand\(\)](#), and [TTTController::TTTController\(\)](#).

6.5.3.6 validateCommand()

```
bool HumanPlayer::validateCommand (
    const QString & command ) [private]
```

Validates whether the recognized voice command corresponds to a valid move.

Verifies that the command is understandable (e.g., "play cell 5") and that the indicated cell is available on the board.

Parameters

<i>command</i>	The text string to validate.
----------------	------------------------------

Returns

true if the command is valid and the cell is available.

6.5.4 Member Data Documentation**6.5.4.1 stt_**

```
SpeechManager* HumanPlayer::stt_ [private]
```

Pointer to the [SpeechManager](#) object, the *worker* for voice recognition.

Referenced by [HumanPlayer\(\)](#).

6.5.4.2 sttThread_

```
QThread* HumanPlayer::sttThread_ [private]
```

Pointer to the worker thread dedicated to running the [SpeechManager](#).

Referenced by [HumanPlayer\(\)](#).

The documentation for this class was generated from the following files:

- [include/AI/HumanPlayer.h](#)
- [src/AI/HumanPlayer.cpp](#)

6.6 MiniMaxAgent Class Reference

The [MiniMaxAgent](#) class implements the logic to select the best move using the Minimax algorithm.

```
#include <MiniMaxAgent.h>
```

Inherits [AIAgent](#).

Public Member Functions

- [MiniMaxAgent](#) (const [Board](#) &board, [BoardMarks](#) Almark, unsigned short depth, QObject *parent=nullptr)
Constructor for the [MiniMaxAgent](#) class.
- void [play](#) () override
Starts the Minimax algorithm to determine and execute the AI's best move.

Public Member Functions inherited from [AIAgent](#)

- [AIAgent](#) ([BoardMarks](#) playerMark, const [Board](#) &borad_ref, QObject *parent_=nullptr)
Constructor for the [AIAgent](#) base class.
- virtual [~AIAgent](#) ()
Virtual destructor.

Public Member Functions inherited from **Player**

- **Player** (const **BoardMarks** &playerMark, const **Board** &board_ref, QObject *parent_=nullptr)
*Constructor for the base **Player** class.*
- virtual ~**Player** ()
Virtual destructor.

Private Member Functions

- **BoardMarks** playerMark_ () const
Returns the human player's marker.
- short **maxMove** (**Board** &board, unsigned short depth, short alpha, short beta) const
Maximizer node function (AI's turn).
- short **minMove** (**Board** &board, unsigned short depth, short alpha, short beta) const
Minimizer node function (human player's turn).
- short **score** (const **BoardState** state) const
Assigns a score to a final game state.

Private Attributes

- const unsigned short **depth_**
Maximum search depth of the Minimax algorithm tree.

Static Private Attributes

- static constexpr short **AI_WIN_SCORE** = 1
Score assigned to the final state when the AI wins the game.
- static constexpr short **PLAYER_WIN_SCORE** = -1
Score assigned to the final state when the human player wins the game.
- static constexpr short **TIE_SCORE** = 0
Score assigned to the final state when the game ends in a tie.

Additional Inherited Members

Signals inherited from **Player**

- void **playerFinished** (int const &cell)
Signal emitted when the player has completed their move.

Protected Attributes inherited from **Player**

- const **Board** * **board_**
Constant pointer to the game board.
- const **BoardMarks** & **mark_**
Constant reference to the marker assigned to this player (X or O).

6.6.1 Detailed Description

The [MiniMaxAgent](#) class implements the logic to select the best move using the Minimax algorithm.

Inherits from [AIAgent](#) (the base class for AI agents). This agent recursively simulates all possible future moves until reaching a final state or the maximum search depth (`depth_`). Uses **Alpha-Beta pruning** to optimize the search. The algorithm evaluates final states by assigning scores: positive for AI victory, negative for human player victory, and zero for a tie.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 MiniMaxAgent()

```
MiniMaxAgent::MiniMaxAgent (
    const Board & board,
    BoardMarks AImark,
    unsigned short depth,
    QObject * parent = nullptr ) [explicit]
```

Constructor for the [MiniMaxAgent](#) class.

Initializes the agent with the board, the marker assigned to the AI, and the maximum depth for the Minimax search.

Parameters

<i>board</i>	Constant reference to the game board.
<i>AImark</i>	The marker (BoardMarks::X or BoardMarks::O) assigned to the AI.
<i>depth</i>	The maximum search depth (cutoff) for the Minimax algorithm.
<i>parent</i>	Pointer to the parent QObject.

6.6.3 Member Function Documentation

6.6.3.1 maxMove()

```
short MiniMaxAgent::maxMove (
    Board & board,
    unsigned short depth,
    short alpha,
    short beta ) const [private]
```

Maximizer node function (AI's turn).

Simulates the AI's turn, choosing the move that maximizes the final score and applying Alpha-Beta pruning.

Parameters

<i>board</i>	Reference to the board model (temporarily modified during simulation).
<i>depth</i>	The current depth of the search tree.
<i>alpha</i>	The minimum value guaranteed for the maximizer (AI).
<i>beta</i>	The maximum value guaranteed for the minimizer (Player).

Returns

The maximum possible score for the current state.

References [Board::at\(\)](#), [Empty](#), [Board::evaluateBoard\(\)](#), [Player::mark_](#), [minMove\(\)](#), [NoWinner](#), [Board::resetCell\(\)](#), [score\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

Referenced by [minMove\(\)](#).

6.6.3.2 minMove()

```
short MiniMaxAgent::minMove (
    Board & board,
    unsigned short depth,
    short alpha,
    short beta ) const [private]
```

Minimizer node function (human player's turn).

Simulates the human player's turn, choosing the move that minimizes the AI's final score and applying Alpha-Beta pruning.

Parameters

<i>board</i>	Reference to the board model (temporarily modified during simulation).
<i>depth</i>	The current depth of the search tree.
<i>alpha</i>	The minimum value guaranteed for the maximizer (AI).
<i>beta</i>	The maximum value guaranteed for the minimizer (Player).

Returns

The minimum possible score for the current state.

References [Board::at\(\)](#), [Empty](#), [Board::evaluateBoard\(\)](#), [maxMove\(\)](#), [NoWinner](#), [playerMark_\(\)](#), [Board::resetCell\(\)](#), [score\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

Referenced by [maxMove\(\)](#), and [play\(\)](#).

6.6.3.3 play()

```
void MiniMaxAgent::play ( ) [override], [virtual]
```

Starts the Minimax algorithm to determine and execute the AI's best move.

This method is the implementation of the abstract function from the [Player](#) class. Calculates the best cell and emits the `playerFinished` signal with the index of the chosen cell.

Implements [Player](#).

References [Board::at\(\)](#), [Player::board_](#), [depth_](#), [Empty](#), [Board::evaluateBoard\(\)](#), [Player::mark_](#), [minMove\(\)](#), [NoWinner](#), [Player::playerFinished\(\)](#), [Board::resetCell\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

6.6.3.4 playerMark_()

```
BoardMarks MiniMaxAgent::playerMark_ ( ) const [inline], [private]
```

Returns the human player's marker.

Automatically determines the opponent player's marker based on the AI's marker.

Returns

The marker ([BoardMarks::X](#) or [BoardMarks::O](#)) of the human player.

References [Player::mark_](#), [O](#), and [X](#).

Referenced by [minMove\(\)](#).

6.6.3.5 score()

```
short MiniMaxAgent::score (
    const BoardState state ) const [private]
```

Assigns a score to a final game state.

Evaluates the board state by determining if there is a winner, a loser, or a tie.

Parameters

<i>state</i>	The final game state (BoardState).
--------------	------------------------------------

Returns

A score based on the game state ([AI_WIN_SCORE](#), [PLAYER_WIN_SCORE](#), [TIE_SCORE](#)).

References [AI_WIN_SCORE](#), [Player::mark_](#), [O](#), [OWins](#), [PLAYER_WIN_SCORE](#), [TIE_SCORE](#), [X](#), and [XWins](#).

Referenced by [maxMove\(\)](#), and [minMove\(\)](#).

6.6.4 Member Data Documentation

6.6.4.1 AI_WIN_SCORE

```
constexpr short MiniMaxAgent::AI_WIN_SCORE = 1 [static], [constexpr], [private]
```

Score assigned to the final state when the AI wins the game.

Referenced by [score\(\)](#).

6.6.4.2 depth_

```
const unsigned short MiniMaxAgent::depth_ [private]
```

Maximum search depth of the Minimax algorithm tree.

Defines the limit of how many future moves the agent will simulate.

Referenced by [play\(\)](#).

6.6.4.3 PLAYER_WIN_SCORE

```
constexpr short MiniMaxAgent::PLAYER_WIN_SCORE = -1 [static], [constexpr], [private]
```

Score assigned to the final state when the human player wins the game.

Referenced by [score\(\)](#).

6.6.4.4 TIE_SCORE

```
constexpr short MiniMaxAgent::TIE_SCORE = 0 [static], [constexpr], [private]
```

Score assigned to the final state when the game ends in a tie.

Referenced by [score\(\)](#).

The documentation for this class was generated from the following files:

- [include/AI/MiniMaxAgent.h](#)
- [src/AI/MiniMaxAgent.cpp](#)

6.7 NetworkManager Class Reference

TCP communication manager class.

```
#include <NetworkManager.h>
```

Inherits QObject.

Signals

- void [connected](#) ()
Signal emitted when the TCP connection is successfully established.
- void [disconnected](#) ()
Signal emitted when the TCP connection is closed (by host, server or error).
- void [errorOccurred](#) (const QString &error)
Signal emitted when an error occurs during connection or transmission.
- void [messageSent](#) (const QString &message)
Signal emitted after a message has been sent to the server.
- void [messageReceived](#) (const QString &message)
Signal emitted when a complete message has been received from the server.

Public Member Functions

- [NetworkManager](#) (QObject *parent=nullptr)
Constructor of the [NetworkManager](#) class.
- void [connectToServer](#) (const QString &ip, quint16 port)
Attempts to establish a connection with a remote server.
- void [sendMessage](#) (const QString &message)
Sends a text message to the connected server.

Private Slots

- void [onConnected](#) ()
Slot invoked when the QTcpSocket connects successfully.
- void [onDisconnected](#) ()
Slot invoked when the QTcpSocket disconnects.
- void [onError](#) (QAbstractSocket::SocketError socketError)
Slot invoked when the QTcpSocket reports an error.
- void [onSocketReadyRead](#) ()
Slot invoked when there is new data ready to read in the socket.

Private Attributes

- QTcpSocket * [socket_](#)
Pointer to the QTcpSocket object that manages the TCP connection.

6.7.1 Detailed Description

TCP communication manager class.

This class provides an interface to connect to a TCP server, send and receive messages asynchronously using QTcpSocket.

Emits signals to notify connection status, errors and message reception.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 NetworkManager()

```
NetworkManager::NetworkManager (
    QObject * parent = nullptr ) [explicit]
```

Constructor of the [NetworkManager](#) class.

Initializes the QTcpSocket.

Parameters

<i>parent</i>	Pointer to the parent object (QObject).
---------------	---

References [onConnected\(\)](#), [onDisconnected\(\)](#), [onError\(\)](#), [onSocketReadyRead\(\)](#), and [socket_](#).

6.7.3 Member Function Documentation

6.7.3.1 connected

```
void NetworkManager::connected ( ) [signal]
```

Signal emitted when the TCP connection is successfully established.

Referenced by [onConnected\(\)](#), and [TTTController::setConnections\(\)](#).

6.7.3.2 connectToServer()

```
void NetworkManager::connectToServer (
    const QString & ip,
    quint16 port )
```

Attempts to establish a connection with a remote server.

Starts the non-blocking connection process. The connection result is notified through the [connected\(\)](#) or [errorOccurred\(\)](#) signals.

Parameters

<i>ip</i>	The IP address or host name of the server (e.g., "127.0.0.1").
<i>port</i>	The TCP port number of the server.

References [socket_](#).

Referenced by [TTTController::setConnections\(\)](#).

6.7.3.3 disconnected

```
void NetworkManager::disconnected ( ) [signal]
```

Signal emitted when the TCP connection is closed (by host, server or error).

Referenced by [onDisconnected\(\)](#), and [TTTController::setConnections\(\)](#).

6.7.3.4 errorOccurred

```
void NetworkManager::errorOccurred (
    const QString & error ) [signal]
```

Signal emitted when an error occurs during connection or transmission.

Parameters

<i>error</i>	Description of the error.
--------------	---------------------------

Referenced by [sendMessage\(\)](#).

6.7.3.5 messageReceived

```
void NetworkManager::messageReceived (
    const QString & message ) [signal]
```

Signal emitted when a complete message has been received from the server.

Parameters

<i>message</i>	The received text message.
----------------	----------------------------

Referenced by [onSocketReadyRead\(\)](#), and [TTTController::setConnections\(\)](#).

6.7.3.6 messageSent

```
void NetworkManager::messageSent (
    const QString & message ) [signal]
```

Signal emitted after a message has been sent to the server.

Parameters

<i>message</i>	The text message that was sent.
----------------	---------------------------------

Referenced by [sendMessage\(\)](#), and [TTTController::setConnections\(\)](#).

6.7.3.7 onConnected

```
void NetworkManager::onConnected ( ) [private], [slot]
```

Slot invoked when the QTcpSocket connects successfully.

Emits the [connected\(\)](#) signal.

References [connected\(\)](#).

Referenced by [NetworkManager\(\)](#).

6.7.3.8 onDisconnected

```
void NetworkManager::onDisconnected ( ) [private], [slot]
```

Slot invoked when the QTcpSocket disconnects.

Emits the [disconnected\(\)](#) signal.

References [disconnected\(\)](#).

Referenced by [NetworkManager\(\)](#).

6.7.3.9 onError

```
void NetworkManager::onError (
    QAbstractSocket::SocketError socketError ) [private], [slot]
```

Slot invoked when the QTcpSocket reports an error.

Maps the socket error to a readable description and emits [errorOccurred\(\)](#).

Parameters

<i>socketError</i>	The socket error code.
--------------------	------------------------

References [socket_](#).

Referenced by [NetworkManager\(\)](#).

6.7.3.10 onSocketReadyRead

```
void NetworkManager::onSocketReadyRead ( ) [private], [slot]
```

Slot invoked when there is new data ready to read in the socket.

Reads all available data and processes it as a message.

References [messageReceived\(\)](#), and [socket_](#).

Referenced by [NetworkManager\(\)](#).

6.7.3.11 sendMessage()

```
void NetworkManager::sendMessage (
    const QString & message )
```

Sends a text message to the connected server.

The message is converted to bytes and written to the socket.

Parameters

<i>message</i>	The text message to send.
----------------	---------------------------

References [errorOccurred\(\)](#), [messageSent\(\)](#), and [socket_](#).

Referenced by [TTTController::updateGame\(\)](#).

6.7.4 Member Data Documentation**6.7.4.1 socket_**

```
QTcpSocket* NetworkManager::socket_ [private]
```

Pointer to the QTcpSocket object that manages the TCP connection.

Referenced by [connectToServer\(\)](#), [NetworkManager\(\)](#), [onError\(\)](#), [onSocketReadyRead\(\)](#), and [sendMessage\(\)](#).

The documentation for this class was generated from the following files:

- include/Network/[NetworkManager.h](#)
- src/Network/[NetworkManager.cpp](#)

6.8 Player Class Reference

The [Player](#) class is an abstract base class for different types of players.

```
#include <Player.h>
```

Inherits QObject.

Inherited by [AIAgent](#), and [HumanPlayer](#).

Signals

- void [playerFinished](#) (int const &cell)
Signal emitted when the player has completed their move.

Public Member Functions

- [Player](#) (const [BoardMarks](#) &playerMark, const [Board](#) &board_ref, QObject *parent_=nullptr)
Constructor for the base [Player](#) class.
- virtual [~Player](#) ()
Virtual destructor.
- virtual void [play](#) ()=0
Pure abstract method to start the player's move logic.

Protected Attributes

- const [Board](#) * [board_](#)
Constant pointer to the game board.
- const [BoardMarks](#) & [mark_](#)
Constant reference to the marker assigned to this player (X or O).

6.8.1 Detailed Description

The [Player](#) class is an abstract base class for different types of players.

This class defines the common interface for all participants in the Tic-Tac-Toe game, whether they are human players ([HumanPlayer](#)), AI agents ([MiniMaxAgent](#)), or network players. It provides the mechanism to start the turn ([play\(\)](#)) and notify the completed move.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 Player()

```
Player::Player (
    const BoardMarks & playerMark,
    const Board & board_ref,
    QObject * parent_ = nullptr ) [inline], [explicit]
```

Constructor for the base [Player](#) class.

Initializes the player with their marker (X or O) and a reference to the game board so the player can query its state.

Parameters

<i>playerMark</i>	The marker (BoardMarks::X or BoardMarks::O) assigned to the player.
<i>board_ref</i>	Constant reference to the game Board object (Model).
<i>parent_</i>	Pointer to the parent QObject (nullptr by default).

6.8.2.2 ~Player()

```
virtual Player::~~Player ( ) [inline], [virtual]
```

Virtual destructor.

Allows proper destruction of derived classes through a base class pointer.

6.8.3 Member Function Documentation

6.8.3.1 play()

```
virtual void Player::play ( ) [pure virtual]
```

Pure abstract method to start the player's move logic.

Derived classes must implement this method to execute their turn logic (e.g., wait for voice/click input, calculate the best AI move).

Implemented in [HumanPlayer](#), and [MiniMaxAgent](#).

6.8.3.2 playerFinished

```
void Player::playerFinished (
    int const & cell ) [signal]
```

Signal emitted when the player has completed their move.

This signal should be emitted by the player after determining their move to notify the controller which cell should be marked.

Parameters

<i>cell</i>	The index of the cell (0-8) chosen by the player.
-------------	---

Referenced by [HumanPlayer::parseSpeechCommand\(\)](#), [MiniMaxAgent::play\(\)](#), and [TTTController::setConnections\(\)](#).

6.8.4 Member Data Documentation

6.8.4.1 board_

```
const Board* Player::board_ [protected]
```

Constant pointer to the game board.

Used to query the current state of cells before making a move.

Referenced by [HumanPlayer::parseSpeechCommand\(\)](#), and [MiniMaxAgent::play\(\)](#).

6.8.4.2 mark_

```
const BoardMarks& Player::mark_ [protected]
```

Constant reference to the marker assigned to this player (X or O).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::play\(\)](#), [MiniMaxAgent::playerMark_\(\)](#), and [MiniMaxAgent::score\(\)](#).

The documentation for this class was generated from the following file:

- [include/AI/Player.h](#)

6.9 SpeechManager Class Reference

Voice recognition manager class using Pocketsphinx and PortAudio.

```
#include <SpeechManager.h>
```

Inherits QObject.

Signals

- void [speechRecognized](#) (const QString &text)
Signal emitted when a phrase or command has been recognized.
- void [speechReadyToProcess](#) (const char *hyp)
Signal emitted when Pocketsphinx returns a new hypothesis.

Public Member Functions

- [SpeechManager](#) (QObject *parent=nullptr)
Constructor of the [SpeechManager](#) class.
- [~SpeechManager](#) ()
Destructor of the [SpeechManager](#) class.
- void [configure](#) ()
Configures the initial Pocketsphinx parameters.
- void [init](#) ()
Initializes the Pocketsphinx decoder and PortAudio.
- void [startListening](#) ()
Starts audio capture and the voice recognition loop.

Private Slots

- void [processRecognitionResult](#) (const char *hyp)
Processes the hypothesis result returned by Pocketsphinx.

Private Member Functions

- bool [hypIsValid](#) ()
Verifies if the recognition hypothesis is valid.
- void [freeHeap](#) ()
Frees memory allocated on the heap.

Private Attributes

- PaStream * [stream_](#)
Pointer to the PortAudio audio stream.
- PaError [err_](#)
PortAudio error code.
- ps_decoder_t * [decoder_](#)
Main pointer to the Pocketsphinx decoder.
- ps_config_t * [config_](#)
Configuration used by the decoder.
- ps_endpoint_t * [ep_](#)
Pocketsphinx voice activity detector (VAD).
- short * [frame_](#)
Buffer to store audio fragments.
- size_t [frame_size_](#)
Size of audio fragments.
- const char * [hyp_](#)
The current recognition hypothesis.

6.9.1 Detailed Description

Voice recognition manager class using Pocketsphinx and PortAudio.

This class encapsulates all the logic for audio capture (PortAudio) and voice recognition processing (Pocketsphinx) in a separate worker thread.

Emits signals when speech is recognized, allowing safe communication with the main thread.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 SpeechManager()

```
SpeechManager::SpeechManager (
    QObject * parent = nullptr ) [explicit]
```

Constructor of the [SpeechManager](#) class.

Initializes PortAudio and Pocketsphinx resource pointers to nullptr and configures the object for thread management.

Parameters

<i>parent</i>	Pointer to the parent object (QObject).
---------------	---

6.9.2.2 ~SpeechManager()

```
SpeechManager::~~SpeechManager ( )
```

Destructor of the [SpeechManager](#) class.

Ensures the release of all Pocketsphinx and PortAudio resources (`ps_free`, `Pa_Terminate`, etc.) to prevent memory leaks.

References [freeHeap\(\)](#).

6.9.3 Member Function Documentation

6.9.3.1 `configure()`

```
void SpeechManager::configure ( )
```

Configures the initial Pocketsphinx parameters.

This function is responsible for loading language models, the dictionary and acoustic parameters into the `ps_config_t` structure. Must be called before `init()`.

References [config_](#), and [ConfigManager::instance\(\)](#).

Referenced by [HumanPlayer::HumanPlayer\(\)](#).

6.9.3.2 `freeHeap()`

```
void SpeechManager::freeHeap ( ) [private]
```

Frees memory allocated on the heap.

Helper function used by the destructor to free all Pocketsphinx and PortAudio pointers.

References [decoder_](#), and [ep_](#).

Referenced by [~SpeechManager\(\)](#).

6.9.3.3 `hypIsValid()`

```
bool SpeechManager::hypIsValid ( ) [private]
```

Verifies if the recognition hypothesis is valid.

Checks if the hypothesis is not null and meets the minimum requirements of confidence or length to be considered a command.

Returns

true if the hypothesis is valid.

References [hyp_](#).

6.9.3.4 init()

```
void SpeechManager::init ( )
```

Initializes the Pocketsphinx decoder and PortAudio.

Creates and initializes the decoder (`ps_decoder_t`) with the configuration established in `configure()` and prepares PortAudio audio capture.

References `config_`, `decoder_`, `ep_`, `err_`, `frame_`, `frame_size_`, and `stream_`.

Referenced by `startListening()`.

6.9.3.5 processRecognitionResult

```
void SpeechManager::processRecognitionResult (
    const char * hyp ) [private], [slot]
```

Processes the hypothesis result returned by Pocketsphinx.

This slot is responsible for performing final validation and emitting the `speechRecognized` signal if the hypothesis is valid.

Parameters

<i>hyp</i>	The recognized text string.
------------	-----------------------------

References `speechRecognized()`.

6.9.3.6 speechReadyToProcess

```
void SpeechManager::speechReadyToProcess (
    const char * hyp ) [signal]
```

Signal emitted when Pocketsphinx returns a new hypothesis.

Mainly used for internal communication between methods of the worker thread before sending the result to the main thread.

Parameters

<i>hyp</i>	The recognition hypothesis as a C character string.
------------	---

Referenced by `startListening()`.

6.9.3.7 speechRecognized

```
void SpeechManager::speechRecognized (
    const QString & text ) [signal]
```

Signal emitted when a phrase or command has been recognized.

Parameters

<i>text</i>	The recognized text string (the hypothesis).
-------------	--

Referenced by [HumanPlayer::HumanPlayer\(\)](#), and [processRecognitionResult\(\)](#).

6.9.3.8 startListening()

```
void SpeechManager::startListening ( )
```

Starts audio capture and the voice recognition loop.

Opens the PortAudio audio stream and starts the Pocketsphinx continuous voice recognition process. Must be called from the worker thread.

References [decoder_](#), [ep_](#), [err_](#), [frame_](#), [frame_size_](#), [hyp_](#), [init\(\)](#), [speechReadyToProcess\(\)](#), and [stream_](#).

Referenced by [HumanPlayer::HumanPlayer\(\)](#).

6.9.4 Member Data Documentation

6.9.4.1 config_

```
ps_config_t* SpeechManager::config_ [private]
```

Configuration used by the decoder.

Referenced by [configure\(\)](#), and [init\(\)](#).

6.9.4.2 decoder_

```
ps_decoder_t* SpeechManager::decoder_ [private]
```

Main pointer to the Pocketsphinx decoder.

Referenced by [freeHeap\(\)](#), [init\(\)](#), and [startListening\(\)](#).

6.9.4.3 ep_

```
ps_endpointinter_t* SpeechManager::ep_ [private]
```

Pocketsphinx voice activity detector (VAD).

Referenced by [freeHeap\(\)](#), [init\(\)](#), and [startListening\(\)](#).

6.9.4.4 err_

```
PaError SpeechManager::err_ [private]
```

PortAudio error code.

Referenced by [init\(\)](#), and [startListening\(\)](#).

6.9.4.5 frame_

```
short* SpeechManager::frame_ [private]
```

Buffer to store audio fragments.

Referenced by [init\(\)](#), and [startListening\(\)](#).

6.9.4.6 frame_size_

```
size_t SpeechManager::frame_size_ [private]
```

Size of audio fragments.

Referenced by [init\(\)](#), and [startListening\(\)](#).

6.9.4.7 hyp_

```
const char* SpeechManager::hyp_ [private]
```

The current recognition hypothesis.

Referenced by [hyplsValid\(\)](#), and [startListening\(\)](#).

6.9.4.8 stream_

```
PaStream* SpeechManager::stream_ [private]
```

Pointer to the PortAudio audio stream.

Referenced by [init\(\)](#), and [startListening\(\)](#).

The documentation for this class was generated from the following files:

- [include/Speech/SpeechManager.h](#)
- [src/Speech/SpeechManager.cpp](#)

6.10 TicTacToeGame Class Reference

The [TicTacToeGame](#) class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

```
#include <TicTacToeGame.h>
```

Inherits QDialog.

Signals

- void [newGame](#) ()
newGame is emitted when the New Game button is clicked and some reset logic must be executed.
- void [goBack](#) ()
goBack is emitted when the Back button is clicked and the TitleScreen must reappear.

Public Member Functions

- [TicTacToeGame](#) (QWidget *parent=nullptr)
TicTacToeGame constructs a view of a game.
- void [updateCell](#) (Cell &cell, BoardMarks currentPlayer)
updateCell sets the Text of the cell to the current player's mark.
- void [declareGameState](#) (BoardState boardState)
declareGameState outputs the board's state to a QMessageBox.
- vector< Cell > [buildCellButtons](#) (size_t boardSize)
buildCellButtons builds the GUI cells as push buttons.
- void [reset](#) (vector< Cell > &cells)
reset clears out the text of all push button cells.
- void [updateConnectionStatus](#) (bool)
Update de visual indicator of the conection status with socket server.
- void [updateRecordingStatus](#) (bool)
- void [updateWaitingStatus](#) (bool)

Private Member Functions

- void [setConnections](#) ()
setConnections handles setting the connection to emit if cells have cliked, and the navigational back button.
- QString [getPlayerText](#) (BoardMarks currentPlayer)
getPlayerText converts a board mark to a QString.
- QString [getPlayerStyleSheet](#) (BoardMarks currentPlayer)
getPlayerStyleSheet gets a stylesheet associated with each player.
- QString [getBoardFinalStateText](#) (BoardState boardState)
getBoardFinalStateText converts a state to a declaration text, that will be used when declaring the final state of a game.
- void [setLedStyle](#) (QLabel *label, const QString &text, const QString &bgColor, const QString &textColor, int fontSize)

Private Attributes

- `std::unique_ptr< Ui::TicTacToeGame > ui`
Activate or deactivate the visual loading indicator.
- `WaitingSpinnerWidget * waiter`
WaitingSpinnerWidget, intended to be an animated wheel when process is waiting server response.

6.10.1 Detailed Description

The `TicTacToeGame` class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 TicTacToeGame()

```
TicTacToeGame::TicTacToeGame (
    QWidget * parent = nullptr ) [explicit]
```

`TicTacToeGame` constructs a view of a game.

Parameters

<code>parent</code>	is a QWidget type parent that owns this instance of the controller.
---------------------	---

References `setConnections()`, `ui`, `updateConnectionStatus()`, `updateWaitingStatus()`, and `waiter`.

6.10.3 Member Function Documentation

6.10.3.1 buildCellButtons()

```
vector< Cell > TicTacToeGame::buildCellButtons (
    size_t boardSize )
```

`buildCellButtons` builds the GUI cells as push buttons.

Parameters

<code>boardSize</code>	number of rows or columns of a board.
------------------------	---------------------------------------

Returns

a collection of cells containing push button references and their corresponding row and column.

References `defaults::GUI_CELL_COLUMN_SPAN`, `defaults::GUI_CELL_ROW_SPAN`, and `ui`.

Referenced by `TTTController::TTTController()`.

6.10.3.2 declareGameState()

```
void TicTacToeGame::declareGameState (
    BoardState boardState )
```

declareGameState outputs the board's state to a QMessageBox.

Parameters

<i>boardState</i>	the current board state.
-------------------	--------------------------

References [getBoardFinalStateText\(\)](#).

Referenced by [TTTController::updateGameState\(\)](#).

6.10.3.3 getBoardFinalStateText()

```
QString TicTacToeGame::getBoardFinalStateText (
    BoardState boardState ) [private]
```

getBoardFinalStateText converts a state to a declaration text, that will be used when declaring the final state of a game.

Parameters

<i>boardState</i>	the current board state.
-------------------	--------------------------

Returns

QString declaration text, that will be used when declaring the final state of a game.

References [OWins](#), [Tie](#), and [XWins](#).

Referenced by [declareGameState\(\)](#).

6.10.3.4 getPlayerStyleSheet()

```
QString TicTacToeGame::getPlayerStyleSheet (
    BoardMarks currentPlayer ) [private]
```

getPlayerStyleSheet gets a stylesheet associated with each player.

Parameters

<i>currentPlayer</i>	the current player's mark (X or O).
----------------------	-------------------------------------

Returns

a QString stylesheet of the player given in the input.

References [defaults::DEFAULT_COLOR](#), [O](#), [defaults::O_COLOR](#), [X](#), and [defaults::X_COLOR](#).

Referenced by [updateCell\(\)](#).

6.10.3.5 getPlayerText()

```
QString TicTacToeGame::getPlayerText (
    BoardMarks currentPlayer ) [private]
```

getPlayerText converts a board mark to a QString.

Parameters

<i>currentPlayer</i>	the current player's mark (X or O).
----------------------	-------------------------------------

Returns

a QString of the player's mark.

References [O](#), and [X](#).

Referenced by [updateCell\(\)](#).

6.10.3.6 goBack

```
void TicTacToeGame::goBack ( ) [signal]
```

goBack is emitted when the Back button is clicked and the TitleScreen must reappear.

Referenced by [TitleScreen::startGame\(\)](#).

6.10.3.7 newGame

```
void TicTacToeGame::newGame ( ) [signal]
```

newGame is emitted when the New Game button is clicked and some reset logic must be executed.

Referenced by [TTTController::setConnections\(\)](#).

6.10.3.8 reset()

```
void TicTacToeGame::reset (
    vector< Cell > & cells )
```

reset clears out the text of all push button cells.

Parameters

<i>cells</i>	a collection of cells containing push button references and their corresponding row and column.
--------------	---

Referenced by [TTTController::reset\(\)](#).

6.10.3.9 setConnections()

```
void TicTacToeGame::setConnections ( ) [private]
```

setConnections handles setting the connection to emit if cells have clicked, and the navigational back button.

References [ui](#).

Referenced by [TicTacToeGame\(\)](#).

6.10.3.10 setLedStyle()

```
void TicTacToeGame::setLedStyle (
    QLabel * label,
    const QString & text,
    const QString & bgColor,
    const QString & textColor,
    int fontSize ) [private]
```

Referenced by [updateRecordingStatus\(\)](#), and [updateWaitingStatus\(\)](#).

6.10.3.11 updateCell()

```
void TicTacToeGame::updateCell (
    Cell & cell,
    BoardMarks currentPlayer )
```

updateCell sets the Text of the cell to the current player's mark.

Parameters

<i>cell</i>	a refernece to the clicked cell.
<i>currentPlayer</i>	the current player's mark (X or O).

References [Cell::cellBtn](#), [getPlayerStyleSheet\(\)](#), and [getPlayerText\(\)](#).

Referenced by [TTTController::updateGameState\(\)](#).

6.10.3.12 updateConnectionStatus()

```
void TicTacToeGame::updateConnectionStatus (
    bool isConnected )
```

Update de visual indicator of the conection status with socket server.

References [ui](#).

Referenced by [TicTacToeGame\(\)](#).

6.10.3.13 updateRecordingStatus()

```
void TicTacToeGame::updateRecordingStatus (
    bool isRecording )
```

References [setLedStyle\(\)](#), and [ui](#).

6.10.3.14 updateWaitingStatus()

```
void TicTacToeGame::updateWaitingStatus (
    bool isWaiting )
```

References [setLedStyle\(\)](#), [WaitingSpinnerWidget::start\(\)](#), [WaitingSpinnerWidget::stop\(\)](#), [ui](#), and [waiter](#).

Referenced by [TicTacToeGame\(\)](#).

6.10.4 Member Data Documentation

6.10.4.1 ui

```
std::unique_ptr<Ui::TicTacToeGame> TicTacToeGame::ui [private]
```

Activate or deactivate the visual loading indicator.

ui is a reference to the ui object of the tic tac toe game.

Referenced by [buildCellButtons\(\)](#), [setConnections\(\)](#), [TicTacToeGame\(\)](#), [updateConnectionStatus\(\)](#), [updateRecordingStatus\(\)](#), and [updateWaitingStatus\(\)](#).

6.10.4.2 waiter

```
WaitingSpinnerWidget* TicTacToeGame::waiter [private]
```

[WaitingSpinnerWidget](#), intended to be an animated wheel when process is waiting server response.

Referenced by [TicTacToeGame\(\)](#), and [updateWaitingStatus\(\)](#).

The documentation for this class was generated from the following files:

- include/View/[TicTacToeGame.h](#)
- src/View/[TicTacToeGame.cpp](#)

6.11 TitleScreen Class Reference

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

```
#include <TitleScreen.h>
```

Inherits QMainWindow.

Public Slots

- void [updateSinglePlayer](#) (bool checked)
updateSinglePlayer updates the AI existence flag to true as there is one player.
- void [updateTwoPlayers](#) (bool checked)
updateTwoPlayers updates the AI existence flag to false as there are two player.
- void [updateAIstartsGame](#) (bool checked)
updateAIstartsGame sets the AI starting the first play or not.
- void [updateMiniMaxDepth](#) (int depth)
updateMiniMaxDepth sets the value of the depth cutoff for the minimax algorithm.
- void [startGame](#) ()
startGame hides the title screen and creates a Tic tac toe controller to start a game.

Signals

- void [exited](#) ()
exited is emitted to manually handle the close functionality.

Public Member Functions

- [TitleScreen](#) (QWidget *parent=nullptr)
TitleScreen constructs the view of the title screen.

Private Member Functions

- void [setConnections](#) ()
setConnections sets connections to gather game options from the GUI elements.
- void [closeEvent](#) (QCloseEvent *event) override
closeEvent manually handles the closing event defined in the base class QWidget.

Private Attributes

- std::unique_ptr< Ui::TitleScreen > [ui](#)
ui is a reference to the ui object of the tic tac toe game.
- [TTOptions](#) [options_](#)
options are the values and flags that control a tic tac toe game.

6.11.1 Detailed Description

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 TitleScreen()

```
TitleScreen::TitleScreen (
    QWidget * parent = nullptr ) [explicit]
```

[TitleScreen](#) constructs the view of the title screen.

Parameters

<i>parent</i>	is a QWidget type parent that owns this instance of the controller.
---------------	---

References [setConnections\(\)](#), and [ui](#).

6.11.3 Member Function Documentation

6.11.3.1 closeEvent()

```
void TitleScreen::closeEvent (
    QCloseEvent * event ) [override], [private]
```

`closeEvent` manually handles the closing event defined in the base class `QWidget`.

Parameters

<i>event</i>	the closing event.
--------------	--------------------

References [exited\(\)](#).

6.11.3.2 exited

```
void TitleScreen::exited ( ) [signal]
```

`exited` is emitted to manually handle the close functionality.

Referenced by [closeEvent\(\)](#), and [main\(\)](#).

6.11.3.3 setConnections()

```
void TitleScreen::setConnections ( ) [private]
```

setConnections sets connections to gather game options from the GUI elements.

References [startGame\(\)](#), [ui](#), [updateAIstartsGame\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

Referenced by [TitleScreen\(\)](#).

6.11.3.4 startGame

```
void TitleScreen::startGame ( ) [slot]
```

startGame hides the title screen and creates a Tic tac toe controller to start a game.

References [TicTacToeGame::goBack\(\)](#), and [options_](#).

Referenced by [setConnections\(\)](#).

6.11.3.5 updateAIstartsGame

```
void TitleScreen::updateAIstartsGame (
    bool checked ) [slot]
```

updateAIstartsGame sets the AI starting the first play or not.

Parameters

<i>checked</i>	wether the checkbox is checked or not.
----------------	--

References [TTOptions::AIstarts](#), and [options_](#).

Referenced by [setConnections\(\)](#).

6.11.3.6 updateMiniMaxDepth

```
void TitleScreen::updateMiniMaxDepth (
    int depth ) [slot]
```

updateMiniMaxDepth sets the value of the depth cutoff for the minimax algorithm.

Parameters

<i>depth</i>	the depth cutoff for the minimax algorithm.
--------------	---

References [TTOptions::miniMaxDepth](#), [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

6.11.3.7 updateSinglePlayer

```
void TitleScreen::updateSinglePlayer (
    bool checked ) [slot]
```

updateSinglePlayer updates the AI existence flag to true as there is one player.

Parameters

<i>checked</i>	wether the radio button is checked or not.
----------------	--

References [TTOptions::Alopponent](#), [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

6.11.3.8 updateTwoPlayers

```
void TitleScreen::updateTwoPlayers (
    bool checked ) [slot]
```

updateTwoPlayers updates the AI existence flag to false as there are two player.

Parameters

<i>checked</i>	wether the radio button is checked or not.
----------------	--

References [TTOptions::Alopponent](#), [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

6.11.4 Member Data Documentation

6.11.4.1 options_

```
TTOptions TitleScreen::options_ [private]
```

options are the values and flags that control a tic tac toe game.

Referenced by [startGame\(\)](#), [updateAStartsGame\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

6.11.4.2 ui

```
std::unique_ptr<Ui::TitleScreen> TitleScreen::ui [private]
```

ui is a reference to the ui object of the tic tac toe game.

Referenced by [setConnections\(\)](#), [TitleScreen\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

The documentation for this class was generated from the following files:

- include/View/[TitleScreen.h](#)
- src/View/[TitleScreen.cpp](#)

6.12 TTTController Class Reference

The [TTTController](#) class acts as the controller (C in MVC) for a Tic-Tac-Toe game.

```
#include <TTTController.h>
```

Inherits [QObject](#).

Public Slots

- void [updateGame](#) ([Cell](#) &cell)
Slot invoked when a cell has been clicked by a human player.

Signals

- void [sendMessage](#) (const [QString](#) &msg)
Signal emitted to send a message through the network (if the mode is multiplayer).
- void [playTurn](#) ()
Signal emitted to indicate to players to start their turn.

Public Member Functions

- [TTTController](#) (const [TTTOptions](#) &options, [QObject](#) *parent=nullptr)
Constructor of the game controller.
- virtual void [startGame](#) ()
Starts the graphical interface execution and the game.
- [QWidget](#) * [getView](#) ()
Returns a pointer to the QWidget representing the game view.

Protected Member Functions

- void [updateGameState](#) ([Cell](#) &cell)
Updates the game state after a valid move.
- void [switchPlayer](#) ()
Switches the current player marker (from X to O or from O to X).

Protected Attributes

- [TicTacToeGame](#) [view_](#)
The view (View) of the Tic-Tac-Toe game.
- [Board](#) [board_](#)
The model (Model) representing the underlying board state.
- [vector](#)< [Cell](#) > [cells_](#)
Collection of references to the visual elements (cell buttons) of the view.
- const [TTTOptions](#) & [options_](#)
Collection of values and flags that control the game mode and options.
- [BoardMarks](#) [currentPlayer_](#)
The marker (BoardMarks) of the player whose turn it is to play.
- [unique_ptr](#)< [Player](#) > [playerX_](#)
Unique pointer to the Player object using the BoardMarks::X marker.
- [unique_ptr](#)< [Player](#) > [playerO_](#)
Unique pointer to the Player object using the BoardMarks::O marker.
- [NetworkManager](#) * [network_](#)
Pointer to the NetworkManager object for network communication purposes (multiplayer).

Private Slots

- void [onNetworkMessageReceived](#) (const QString &msg)
Slot invoked when receiving a message from the [NetworkManager](#).

Private Member Functions

- void [setConnections](#) ()
Configures signal and slot connections between the view (GUI) and the controller.
- void [reset](#) ()
Resets all internal game components.

6.12.1 Detailed Description

The [TTTController](#) class acts as the controller (C in MVC) for a Tic-Tac-Toe game.

[TTTController](#) manages the interaction between the user interface (the View, [TicTacToeGame](#)), the game state (the Model, [Board](#)) and the game logic, including turn management, AI agents ([MiniMaxAgent](#)), human players and network communication ([NetworkManager](#)). It is responsible for receiving game options and orchestrating the game flow.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 TTTController()

```
TTTController::TTTController (
    const TTTOptions & options,
    QObject * parent = nullptr ) [explicit]
```

Constructor of the game controller.

Initializes the model, view, players ([HumanPlayer](#), [MiniMaxAgent](#)) and establishes the initial connections.

Parameters

<i>options</i>	The collection of values and flags that control the game.
<i>parent</i>	Pointer to the parent QObject that owns this instance.

References [TTTOptions::Alopponent](#), [TTTOptions::Alstarts](#), [board_](#), [TTTOptions::boardSize](#), [TicTacToeGame::buildCellButtons\(\)](#), [cells_](#), [TTTOptions::miniMaxDepth](#), [network_](#), [O](#), [options_](#), [playerO_](#), [playerX_](#), [reset\(\)](#), [setConnections\(\)](#), [HumanPlayer::startListening\(\)](#), [HumanPlayer::stopListening\(\)](#), [view_](#), and [X](#).

6.12.3 Member Function Documentation

6.12.3.1 getView()

```
QWidget * TTTController::getView ( ) [inline]
```

Returns a pointer to the QWidget representing the game view.

Returns

Pointer to the view's QWidget object.

References [view_](#).

Referenced by [setConnections\(\)](#).

6.12.3.2 onNetworkMessageReceived

```
void TTTController::onNetworkMessageReceived (
    const QString & msg ) [private], [slot]
```

Slot invoked when receiving a message from the [NetworkManager](#).

Processes the received message (typically a move from a remote player) and updates the game state.

Parameters

<i>msg</i>	The text string received through the network.
------------	---

References [board_](#), [currentPlayer_](#), [Board::evaluateBoard\(\)](#), [NoWinner](#), [playerO_](#), [playerX_](#), [reset\(\)](#), [switchPlayer\(\)](#), and [X](#).

Referenced by [setConnections\(\)](#).

6.12.3.3 playTurn

```
void TTTController::playTurn ( ) [signal]
```

Signal emitted to indicate to players to start their turn.

This signal can be connected to the `play()` method of players to start the move logic (e.g., wait for human click or calculate AI move).

6.12.3.4 reset()

```
void TTTController::reset ( ) [private]
```

Resets all internal game components.

Resets the model ([Board](#)), the view ([TicTacToeGame](#)) and prepares the players for a new game. If the AI agent should start, it makes the first move.

References [board_](#), [cells_](#), [currentPlayer_](#), [playerX_](#), [Board::reset\(\)](#), [TicTacToeGame::reset\(\)](#), [view_](#), and [X](#).

Referenced by [onNetworkMessageReceived\(\)](#), [setConnections\(\)](#), and [TTTController\(\)](#).

6.12.3.5 sendMessage

```
void TTTController::sendMessage (
    const QString & msg ) [signal]
```

Signal emitted to send a message through the network (if the mode is multiplayer).

Parameters

<i>msg</i>	The message to send (typically a representation of the move).
------------	---

6.12.3.6 setConnections()

```
void TTTController::setConnections ( ) [private]
```

Configures signal and slot connections between the view (GUI) and the controller.

This method establishes the necessary connections to handle GUI events, such as clicks on cells or menu buttons.

References [NetworkManager::connected\(\)](#), [NetworkManager::connectToServer\(\)](#), [NetworkManager::disconnected\(\)](#), [ConfigManager::getInt\(\)](#), [ConfigManager::getValue\(\)](#), [getView\(\)](#), [ConfigManager::instance\(\)](#), [NetworkManager::messageReceived\(\)](#), [NetworkManager::messageSent\(\)](#), [network_](#), [TicTacToeGame::newGame\(\)](#), [onNetworkMessageReceived\(\)](#), [Player::playerFinished\(\)](#), [playerO_](#), [playerX_](#), [reset\(\)](#), and [view_](#).

Referenced by [TTTController\(\)](#).

6.12.3.7 startGame()

```
void TTTController::startGame ( ) [virtual]
```

Starts the graphical interface execution and the game.

Shows the GUI (usually by calling `view_.exec()` or `view_.show()`) and begins the game logic.

References [view_](#).

6.12.3.8 switchPlayer()

```
void TTTController::switchPlayer ( ) [protected]
```

Switches the current player marker (from X to O or from O to X).

References [currentPlayer_](#), [O](#), and [X](#).

Referenced by [onNetworkMessageReceived\(\)](#).

6.12.3.9 updateGame

```
void TTTController::updateGame (
    Cell & cell ) [slot]
```

Slot invoked when a cell has been clicked by a human player.

Receives the clicked cell and executes the `updateGameState` logic if the move is valid and it's a local player's turn.

Parameters

<i>cell</i>	Reference to the clicked cell that the player chooses.
-------------	--

References [board_](#), [Cell::col](#), [currentPlayer_](#), [network_](#), [Cell::row](#), [NetworkManager::sendMessage\(\)](#), [Board::setPlayerInput\(\)](#), [updateGameState\(\)](#), and [X](#).

6.12.3.10 updateGameState()

```
void TTTController::updateGameState (
    Cell & cell ) [protected]
```

Updates the game state after a valid move.

Updates the model, refreshes the view, checks victory/tie conditions and switches to the next player.

Parameters

<i>cell</i>	Reference to the cell that has been selected by the player.
-------------	---

References [board_](#), [currentPlayer_](#), [TicTacToeGame::declareGameState\(\)](#), [Board::evaluateBoard\(\)](#), [NoWinner](#), [Board::printBoard\(\)](#), [TicTacToeGame::updateCell\(\)](#), [view_](#), and [X](#).

Referenced by [updateGame\(\)](#).

6.12.4 Member Data Documentation**6.12.4.1 board_**

```
Board TTTController::board_ [protected]
```

The model (Model) representing the underlying board state.

Referenced by [onNetworkMessageReceived\(\)](#), [reset\(\)](#), [TTTController\(\)](#), [updateGame\(\)](#), and [updateGameState\(\)](#).

6.12.4.2 cells_

```
vector<Cell> TTTController::cells_ [protected]
```

Collection of references to the visual elements (cell buttons) of the view.

Referenced by [reset\(\)](#), and [TTTController\(\)](#).

6.12.4.3 currentPlayer_

```
BoardMarks TTTController::currentPlayer_ [protected]
```

The marker (BoardMarks) of the player whose turn it is to play.

Referenced by [onNetworkMessageReceived\(\)](#), [reset\(\)](#), [switchPlayer\(\)](#), [updateGame\(\)](#), and [updateGameState\(\)](#).

6.12.4.4 network_

```
NetworkManager* TTTController::network_ [protected]
```

Pointer to the [NetworkManager](#) object for network communication purposes (multiplayer).

Referenced by [setConnections\(\)](#), [TTTController\(\)](#), and [updateGame\(\)](#).

6.12.4.5 options_

```
const TTTOptions& TTTController::options_ [protected]
```

Collection of values and flags that control the game mode and options.

Referenced by [TTTController\(\)](#).

6.12.4.6 playerO_

```
unique_ptr<Player> TTTController::playerO_ [protected]
```

Unique pointer to the [Player](#) object using the [BoardMarks::O](#) marker.

Referenced by [onNetworkMessageReceived\(\)](#), [setConnections\(\)](#), and [TTTController\(\)](#).

6.12.4.7 playerX_

```
unique_ptr<Player> TTTController::playerX_ [protected]
```

Unique pointer to the [Player](#) object using the [BoardMarks::X](#) marker.

Referenced by [onNetworkMessageReceived\(\)](#), [reset\(\)](#), [setConnections\(\)](#), and [TTTController\(\)](#).

6.12.4.8 view_

```
TicTacToeGame TTTController::view_ [protected]
```

The view (View) of the Tic-Tac-Toe game.

Referenced by [getView\(\)](#), [reset\(\)](#), [setConnections\(\)](#), [startGame\(\)](#), [TTTController\(\)](#), and [updateGameState\(\)](#).

The documentation for this class was generated from the following files:

- include/Controller/[TTTController.h](#)
- src/Controller/[TTTController.cpp](#)

6.13 TTTOptions Struct Reference

The [TTTOptions](#) struct defines a tic tac toe game options.

```
#include <TTTCommonTypes.h>
```

Public Attributes

- `size_t boardSize = defaults::DEFAULT_BOARD_SIZE`
- `unsigned short miniMaxDepth = defaults::DEFAULT_MINIMAX_DEPTH`
- `bool Alopponent = true`
- `bool Alstarts = false`

6.13.1 Detailed Description

The [TTOptions](#) struct defines a tic tac toe game options.

6.13.2 Member Data Documentation

6.13.2.1 Alopponent

```
bool TTOptions::Alopponent = true
```

Referenced by [TTController::TTController\(\)](#), [TitleScreen::updateSinglePlayer\(\)](#), and [TitleScreen::updateTwoPlayers\(\)](#).

6.13.2.2 Alstarts

```
bool TTOptions::Alstarts = false
```

Referenced by [TTController::TTController\(\)](#), and [TitleScreen::updateAlstartsGame\(\)](#).

6.13.2.3 boardSize

```
size_t TTOptions::boardSize = defaults::DEFAULT_BOARD_SIZE
```

Referenced by [TTController::TTController\(\)](#).

6.13.2.4 miniMaxDepth

```
unsigned short TTOptions::miniMaxDepth = defaults::DEFAULT_MINIMAX_DEPTH
```

Referenced by [TTController::TTController\(\)](#), and [TitleScreen::updateMiniMaxDepth\(\)](#).

The documentation for this struct was generated from the following file:

- `include/TTCommonTypes.h`

6.14 WaitingSpinnerWidget Class Reference

```
#include <waitingspinnerwidget.h>
```

Inherits [QWidget](#).

Public Slots

- void [start](#) ()
- void [stop](#) ()

Public Member Functions

- [WaitingSpinnerWidget](#) (QWidget *parent=0, bool centerOnParent=true, bool disableParentWhenSpinning=true)
- [WaitingSpinnerWidget](#) (Qt::WindowModality modality, QWidget *parent=0, bool centerOnParent=true, bool disableParentWhenSpinning=true)
- void [setColor](#) (QColor color)
- void [setRoundness](#) (qreal roundness)
- void [setMinimumTrailOpacity](#) (qreal minimumTrailOpacity)
- void [setTrailFadePercentage](#) (qreal trail)
- void [setRevolutionsPerSecond](#) (qreal revolutionsPerSecond)
- void [setNumberOfLines](#) (int lines)
- void [setLineLength](#) (int length)
- void [setLineWidth](#) (int width)
- void [setInnerRadius](#) (int radius)
- void [setText](#) (QString text)
- QColor [color](#) ()
- qreal [roundness](#) ()
- qreal [minimumTrailOpacity](#) ()
- qreal [trailFadePercentage](#) ()
- qreal [revolutionsPersSecond](#) ()
- int [numberOfLines](#) ()
- int [lineLength](#) ()
- int [lineWidth](#) ()
- int [innerRadius](#) ()
- bool [isSpinning](#) () const

Protected Member Functions

- void [paintEvent](#) (QPaintEvent *paintEvent)

Private Slots

- void [rotate](#) ()

Private Member Functions

- void [initialize](#) ()
- void [updateSize](#) ()
- void [updateTimer](#) ()
- void [updatePosition](#) ()
- [WaitingSpinnerWidget](#) (const [WaitingSpinnerWidget](#) &)
- [WaitingSpinnerWidget](#) & [operator=](#) (const [WaitingSpinnerWidget](#) &)

Static Private Member Functions

- static int [lineCountDistanceFromPrimary](#) (int current, int primary, int totalNrOfLines)
- static QColor [currentLineColor](#) (int distance, int totalNrOfLines, qreal trailFadePerc, qreal minOpacity, QColor color)

Private Attributes

- QColor [_color](#)
- qreal [_roundness](#)
- qreal [_minimumTrailOpacity](#)
- qreal [_trailFadePercentage](#)
- qreal [_revolutionsPerSecond](#)
- int [_numberOfLines](#)
- int [_lineLength](#)
- int [_lineWidth](#)
- int [_innerRadius](#)
- QTimer * [_timer](#)
- bool [_centerOnParent](#)
- bool [_disableParentWhenSpinning](#)
- int [_currentCounter](#)
- bool [_isSpinning](#)

6.14.1 Constructor & Destructor Documentation

6.14.1.1 WaitingSpinnerWidget() [1/3]

```
WaitingSpinnerWidget::WaitingSpinnerWidget (  
    QWidget * parent = 0,  
    bool centerOnParent = true,  
    bool disableParentWhenSpinning = true )
```

Constructor for "standard" widget behaviour - use this constructor if you wish to, e.g. embed your widget in another.

References [initialize\(\)](#).

6.14.1.2 WaitingSpinnerWidget() [2/3]

```
WaitingSpinnerWidget::WaitingSpinnerWidget (  
    Qt::WindowModality modality,  
    QWidget * parent = 0,  
    bool centerOnParent = true,  
    bool disableParentWhenSpinning = true )
```

Constructor - use this constructor to automatically create a modal ("blocking") spinner on top of the calling widget/window. If a valid parent widget is provided, "centreOnParent" will ensure that QtWaitingSpinner automatically centres itself on it, if not, "centreOnParent" is ignored.

References [initialize\(\)](#).

6.14.1.3 WaitingSpinnerWidget() [3/3]

```
WaitingSpinnerWidget::WaitingSpinnerWidget (
    const WaitingSpinnerWidget & ) [private]
```

6.14.2 Member Function Documentation

6.14.2.1 color()

```
QColor WaitingSpinnerWidget::color ( )
```

References [_color](#).

Referenced by [currentLineColor\(\)](#), [paintEvent\(\)](#), and [setColor\(\)](#).

6.14.2.2 currentLineColor()

```
QColor WaitingSpinnerWidget::currentLineColor (
    int distance,
    int totalNrOfLines,
    qreal trailFadePerc,
    qreal minOpacity,
    QColor color ) [static], [private]
```

References [color\(\)](#).

Referenced by [paintEvent\(\)](#).

6.14.2.3 initialize()

```
void WaitingSpinnerWidget::initialize ( ) [private]
```

References [_color](#), [_currentCounter](#), [_innerRadius](#), [_isSpinning](#), [_lineLength](#), [_lineWidth](#), [_minimumTrailOpacity](#), [_numberOfLines](#), [_revolutionsPerSecond](#), [_roundness](#), [_timer](#), [_trailFadePercentage](#), [rotate\(\)](#), [updateSize\(\)](#), and [updateTimer\(\)](#).

Referenced by [WaitingSpinnerWidget\(\)](#), and [WaitingSpinnerWidget\(\)](#).

6.14.2.4 innerRadius()

```
int WaitingSpinnerWidget::innerRadius ( )
```

References [_innerRadius](#).

6.14.2.5 isSpinning()

```
bool WaitingSpinnerWidget::isSpinning ( ) const
```

References [_isSpinning](#).

6.14.2.6 lineCountDistanceFromPrimary()

```
int WaitingSpinnerWidget::lineCountDistanceFromPrimary (
    int current,
    int primary,
    int totalNrOfLines ) [static], [private]
```

Referenced by [paintEvent\(\)](#).

6.14.2.7 lineLength()

```
int WaitingSpinnerWidget::lineLength ( )
```

References [_lineLength](#).

6.14.2.8 lineWidth()

```
int WaitingSpinnerWidget::lineWidth ( )
```

References [_lineWidth](#).

6.14.2.9 minimumTrailOpacity()

```
qreal WaitingSpinnerWidget::minimumTrailOpacity ( )
```

References [_minimumTrailOpacity](#).

Referenced by [setMinimumTrailOpacity\(\)](#).

6.14.2.10 numberOfLines()

```
int WaitingSpinnerWidget::numberOfLines ( )
```

References [_numberOfLines](#).

6.14.2.11 operator=()

```
WaitingSpinnerWidget & WaitingSpinnerWidget::operator= (
    const WaitingSpinnerWidget & ) [private]
```

6.14.2.12 paintEvent()

```
void WaitingSpinnerWidget::paintEvent (
    QPaintEvent * paintEvent ) [protected]
```

References [_color](#), [_currentCounter](#), [_innerRadius](#), [_lineLength](#), [_lineWidth](#), [_minimumTrailOpacity](#), [_numberOfLines](#), [_roundness](#), [_trailFadePercentage](#), [color\(\)](#), [currentLineColor\(\)](#), [lineCountDistanceFromPrimary\(\)](#), and [updatePosition\(\)](#).

6.14.2.13 revolutionsPersSecond()

```
qreal WaitingSpinnerWidget::revolutionsPersSecond ( )
```

References [_revolutionsPerSecond](#).

6.14.2.14 rotate

```
void WaitingSpinnerWidget::rotate ( ) [private], [slot]
```

References [_currentCounter](#), and [_numberOfLines](#).

Referenced by [initialize\(\)](#).

6.14.2.15 roundness()

```
qreal WaitingSpinnerWidget::roundness ( )
```

References [_roundness](#).

Referenced by [setRoundness\(\)](#).

6.14.2.16 setColor()

```
void WaitingSpinnerWidget::setColor (
    QColor color )
```

References [_color](#), and [color\(\)](#).

6.14.2.17 setInnerRadius()

```
void WaitingSpinnerWidget::setInnerRadius (
    int radius )
```

References [_innerRadius](#), and [updateSize\(\)](#).

6.14.2.18 setLineLength()

```
void WaitingSpinnerWidget::setLineLength (
    int length )
```

References [_lineLength](#), and [updateSize\(\)](#).

6.14.2.19 setLineWidth()

```
void WaitingSpinnerWidget::setLineWidth (
    int width )
```

References [_lineWidth](#), and [updateSize\(\)](#).

6.14.2.20 setMinimumTrailOpacity()

```
void WaitingSpinnerWidget::setMinimumTrailOpacity (
    qreal minimumTrailOpacity )
```

References [_minimumTrailOpacity](#), and [minimumTrailOpacity\(\)](#).

6.14.2.21 setNumberOfLines()

```
void WaitingSpinnerWidget::setNumberOfLines (
    int lines )
```

References [_currentCounter](#), [_numberOfLines](#), and [updateTimer\(\)](#).

6.14.2.22 setRevolutionsPerSecond()

```
void WaitingSpinnerWidget::setRevolutionsPerSecond (
    qreal revolutionsPerSecond )
```

References [_revolutionsPerSecond](#), and [updateTimer\(\)](#).

6.14.2.23 setRoundness()

```
void WaitingSpinnerWidget::setRoundness (
    qreal roundness )
```

References [_roundness](#), and [roundness\(\)](#).

6.14.2.24 setText()

```
void WaitingSpinnerWidget::setText (
    QString text )
```

6.14.2.25 setTrailFadePercentage()

```
void WaitingSpinnerWidget::setTrailFadePercentage (
    qreal trail )
```

References [_trailFadePercentage](#).

6.14.2.26 start

```
void WaitingSpinnerWidget::start ( ) [slot]
```

References [_currentCounter](#), [_disableParentWhenSpinning](#), [_isSpinning](#), [_timer](#), and [updatePosition\(\)](#).

Referenced by [TicTacToeGame::updateWaitingStatus\(\)](#).

6.14.2.27 stop

```
void WaitingSpinnerWidget::stop ( ) [slot]
```

References [_currentCounter](#), [_disableParentWhenSpinning](#), [_isSpinning](#), and [_timer](#).

Referenced by [TicTacToeGame::updateWaitingStatus\(\)](#).

6.14.2.28 trailFadePercentage()

```
qreal WaitingSpinnerWidget::trailFadePercentage ( )
```

References [_trailFadePercentage](#).

6.14.2.29 updatePosition()

```
void WaitingSpinnerWidget::updatePosition ( ) [private]
```

References [_centerOnParent](#).

Referenced by [paintEvent\(\)](#), and [start\(\)](#).

6.14.2.30 updateSize()

```
void WaitingSpinnerWidget::updateSize ( ) [private]
```

References [_innerRadius](#), and [_lineLength](#).

Referenced by [initialize\(\)](#), [setInnerRadius\(\)](#), [setLineLength\(\)](#), and [setLineWidth\(\)](#).

6.14.2.31 updateTimer()

```
void WaitingSpinnerWidget::updateTimer ( ) [private]
```

References [_numberOfLines](#), [_revolutionsPerSecond](#), and [_timer](#).

Referenced by [initialize\(\)](#), [setNumberOfLines\(\)](#), and [setRevolutionsPerSecond\(\)](#).

6.14.3 Member Data Documentation

6.14.3.1 _centerOnParent

```
bool WaitingSpinnerWidget::_centerOnParent [private]
```

Referenced by [updatePosition\(\)](#).

6.14.3.2 `_color`

```
QColor WaitingSpinnerWidget::_color [private]
```

Referenced by [color\(\)](#), [initialize\(\)](#), [paintEvent\(\)](#), and [setColor\(\)](#).

6.14.3.3 `_currentCounter`

```
int WaitingSpinnerWidget::_currentCounter [private]
```

Referenced by [initialize\(\)](#), [paintEvent\(\)](#), [rotate\(\)](#), [setNumberOfLines\(\)](#), [start\(\)](#), and [stop\(\)](#).

6.14.3.4 `_disableParentWhenSpinning`

```
bool WaitingSpinnerWidget::_disableParentWhenSpinning [private]
```

Referenced by [start\(\)](#), and [stop\(\)](#).

6.14.3.5 `_innerRadius`

```
int WaitingSpinnerWidget::_innerRadius [private]
```

Referenced by [initialize\(\)](#), [innerRadius\(\)](#), [paintEvent\(\)](#), [setInnerRadius\(\)](#), and [updateSize\(\)](#).

6.14.3.6 `_isSpinning`

```
bool WaitingSpinnerWidget::_isSpinning [private]
```

Referenced by [initialize\(\)](#), [isSpinning\(\)](#), [start\(\)](#), and [stop\(\)](#).

6.14.3.7 `_lineLength`

```
int WaitingSpinnerWidget::_lineLength [private]
```

Referenced by [initialize\(\)](#), [lineLength\(\)](#), [paintEvent\(\)](#), [setLineLength\(\)](#), and [updateSize\(\)](#).

6.14.3.8 `_lineWidth`

```
int WaitingSpinnerWidget::_lineWidth [private]
```

Referenced by [initialize\(\)](#), [lineWidth\(\)](#), [paintEvent\(\)](#), and [setLineWidth\(\)](#).

6.14.3.9 `_minimumTrailOpacity`

```
qreal WaitingSpinnerWidget::_minimumTrailOpacity [private]
```

Referenced by [initialize\(\)](#), [minimumTrailOpacity\(\)](#), [paintEvent\(\)](#), and [setMinimumTrailOpacity\(\)](#).

6.14.3.10 `_numberOfLines`

```
int WaitingSpinnerWidget::_numberOfLines [private]
```

Referenced by [initialize\(\)](#), [numberOfLines\(\)](#), [paintEvent\(\)](#), [rotate\(\)](#), [setNumberOfLines\(\)](#), and [updateTimer\(\)](#).

6.14.3.11 `_revolutionsPerSecond`

```
qreal WaitingSpinnerWidget::_revolutionsPerSecond [private]
```

Referenced by [initialize\(\)](#), [revolutionsPersSecond\(\)](#), [setRevolutionsPerSecond\(\)](#), and [updateTimer\(\)](#).

6.14.3.12 `_roundness`

```
qreal WaitingSpinnerWidget::_roundness [private]
```

Referenced by [initialize\(\)](#), [paintEvent\(\)](#), [roundness\(\)](#), and [setRoundness\(\)](#).

6.14.3.13 `_timer`

```
QTimer* WaitingSpinnerWidget::_timer [private]
```

Referenced by [initialize\(\)](#), [start\(\)](#), [stop\(\)](#), and [updateTimer\(\)](#).

6.14.3.14 `_trailFadePercentage`

```
qreal WaitingSpinnerWidget::_trailFadePercentage [private]
```

Referenced by [initialize\(\)](#), [paintEvent\(\)](#), [setTrailFadePercentage\(\)](#), and [trailFadePercentage\(\)](#).

The documentation for this class was generated from the following files:

- include/View/[waitingspinnerwidget.h](#)
- src/[waitingspinnerwidget.cpp](#)

Chapter 7

File Documentation

7.1 docs/plantuml/AIAgent.puml File Reference

7.2 docs/plantuml/Board.puml File Reference

7.3 docs/plantuml/ConfigManager.puml File Reference

7.4 docs/plantuml/HumanPlayer.puml File Reference

7.5 docs/plantuml/MiniMaxAgent.puml File Reference

7.6 docs/plantuml/NetworkManager.puml File Reference

7.7 docs/plantuml/Player.puml File Reference

7.8 docs/plantuml/SpeechManager.puml File Reference

7.9 docs/plantuml/TicTacToeGame.puml File Reference

7.10 docs/plantuml/TitleScreen.puml File Reference

7.11 docs/plantuml/TTTCommonTypes.puml File Reference

7.12 docs/plantuml/TTTController.puml File Reference

7.13 docs/plantuml/WaitingSpinnerWidget.puml File Reference

7.14 include/AI/AIAgent.h File Reference

Definition of the [AIAgent](#) interface for artificial intelligence agents.

```
#include "Player.h"
```

Classes

- class [AIAgent](#)

The [AIAgent](#) interface describes the behavior of any implemented Artificial Intelligence agent.

7.14.1 Detailed Description

Definition of the [AIAgent](#) interface for artificial intelligence agents.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.15 AIAgent.h

[Go to the documentation of this file.](#)

```
00001 #ifndef AIAGENT_H
00002 #define AIAGENT_H
00003 #include "Player.h"
00004
00028 class Board; // Forward declaration
00029
00030 class AIAgent : public Player
00031 {
00032 public:
00042     AIAgent(BoardMarks playerMark, const Board& borad_ref, QObject *parent_ = nullptr)
00043         : Player(playerMark, borad_ref, parent_) {}
00044
00050     virtual ~AIAgent() {}
00051 };
00052
00053 #endif // AIAGENT_H
```

7.16 include/AI/HumanPlayer.h File Reference

Definition of the [HumanPlayer](#) class that represents the human player.

```
#include <Player.h>
#include <SpeechManager.h>
#include <QThread>
```

Classes

- class [HumanPlayer](#)

The [HumanPlayer](#) class represents the human player in the game.

7.16.1 Detailed Description

Definition of the [HumanPlayer](#) class that represents the human player.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.17 HumanPlayer.h

[Go to the documentation of this file.](#)

```

00001 #ifndef HUMANPLAYER_H
00002 #define HUMANPLAYER_H
00003 #include <Player.h>
00004 #include <SpeechManager.h>
00005 #include <QThread>
00006
00045 class HumanPlayer final : public Player{
00046     Q_OBJECT
00047
00048 public:
00059     explicit HumanPlayer(const Board& board_ref, BoardMarks playerMark = BoardMarks::Empty, QObject*
parent = nullptr);
00060
00066     virtual ~HumanPlayer() {}
00067
00074     void play() override;
00075
00076 signals:
00080     void startListening();
00081
00085     void stopListening();
00086
00087 private:
00096     void parseSpeechCommand(const QString &text);
00097
00107     bool validateCommand(const QString &command);
00108
00115     void handleThreadCleanup();
00116
00117     SpeechManager* stt_;
00118     QThread* sttThread_;
00119 };
00120
00121 #endif // HUMANPLAYER_H

```

7.18 include/AI/MiniMaxAgent.h File Reference

Definition of the [MiniMaxAgent](#) class implementing the Minimax algorithm with Alpha-Beta pruning.

```
#include "AIAgent.h"
```

Classes

- class [MiniMaxAgent](#)

The [MiniMaxAgent](#) class implements the logic to select the best move using the Minimax algorithm.

7.18.1 Detailed Description

Definition of the [MiniMaxAgent](#) class implementing the Minimax algorithm with Alpha-Beta pruning.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.19 MiniMaxAgent.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MINIMAXAGENT_H
00002 #define MINIMAXAGENT_H
00003 #include "AIAgent.h"
00004
00040 class MiniMaxAgent final : public AIAgent
00041 {
00042     Q_OBJECT
00043
00044 public:
00056     explicit MiniMaxAgent(const Board& board, BoardMarks AImark, unsigned short depth, QObject* parent
= nullptr);
00057
00064     void play() override;
00065
00066 private:
00072     const unsigned short depth_;
00073
00080     inline BoardMarks playerMark_() const {
00081         return (BoardMarks::X == mark_) ? BoardMarks::O : BoardMarks::X;
00082     }
00083
00087     static constexpr short AI_WIN_SCORE = 1;
00088
00092     static constexpr short PLAYER_WIN_SCORE = -1;
00093
00097     static constexpr short TIE_SCORE = 0;
00098
00111     short maxMove(Board &board, unsigned short depth, short alpha, short beta) const;
00112
00125     short minMove(Board &board, unsigned short depth, short alpha, short beta) const;
00126
00135     short score(const BoardState state) const;
00136 };
00137
00138 #endif // MINIMAXAGENT_H
```

7.20 include/AI/Player.h File Reference

Definition of the [Player](#) abstract base class for different player types.

```
#include <QObject>
#include <TTTCommonTypes.h>
#include <Board.h>
```

Classes

- class [Player](#)

The [Player](#) class is an abstract base class for different types of players.

7.20.1 Detailed Description

Definition of the [Player](#) abstract base class for different player types.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.21 Player.h

[Go to the documentation of this file.](#)

```
00001 #ifndef PLAYER_H
00002 #define PLAYER_H
00003 #include <QObject>
00004 #include <TTTCommonTypes.h>
00005 #include <Board.h>
00006
00039 class Player : public QObject
00040 {
00041     Q_OBJECT
00042 public:
00053     explicit Player(const BoardMarks& playerMark, const Board& board_ref, QObject *parent_ = nullptr)
00054         : QObject(parent_),
00055           board_(&board_ref),
00056           mark_(playerMark) {}
00057
00063     virtual ~Player() {}
00064
00071     virtual void play() = 0;
00072
00073 signals:
00082     void playerFinished(int const& cell);
00083
00084 protected:
00090     const Board* board_;
00091
00095     const BoardMarks& mark_;
00096
00097 };
00098
00099 #endif // PLAYER_H
```

7.22 include/Config/ConfigManager.h File Reference

Definition of the [ConfigManager](#) class for application configuration management.

```
#include <QObject>
#include <QSettings>
#include <QDir>
#include <QDebug>
#include <QCoreApplication>
```

Classes

- class [ConfigManager](#)
The [ConfigManager](#) class manages the application configuration.

7.22.1 Detailed Description

Definition of the [ConfigManager](#) class for application configuration management.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.23 ConfigManager.h

[Go to the documentation of this file.](#)

```
00001 #ifndef CONFIGMANAGER_H
00002 #define CONFIGMANAGER_H
00003
00004 #pragma once
00005 #include <QObject>
00006 #include <QSettings>
00007 #include <QDir>
00008 #include <QDebug>
00009 #include <QCoreApplication>
00010
00042 class ConfigManager final : public QObject
00043 {
00044     Q_OBJECT
00045
00046 public:
00054     static ConfigManager& instance();
00055
00064     void load(const QString& filePath = QString());
00065
00075     QString getValue(const QString& key, const QString& defaultValue = QString()) const;
00076
00086     int getInt(const QString& key, int defaultValue = 0) const;
00087
```

```

00097     bool getBool(const QString& key, bool defaultValue = false) const;
00098
00108     QString resolvePath(const QString& relativePath) const;
00109
00110 private:
00118     explicit ConfigManager(QObject *parent = nullptr);
00119
00125     ~ConfigManager() override = default;
00126
00127     // Block copy and assignment
00128     ConfigManager(const ConfigManager&) = delete;
00129     ConfigManager& operator=(const ConfigManager&) = delete;
00130
00131     QSettings* settings_ = nullptr;
00132     QString baseDir_;
00133 };
00134
00135 #endif // CONFIGMANAGER_H

```

7.24 include/Controller/TTTController.h File Reference

Definition of the [TTTController](#) class for the Tic-Tac-Toe game controller (MVC pattern)

```

#include "MiniMaxAgent.h"
#include "HumanPlayer.h"
#include "Board.h"
#include "TTTCommonTypes.h"
#include "TicTacToeGame.h"
#include "NetworkManager.h"
#include <QTcpSocket>
#include <QObject>
#include <memory>
#include <vector>
#include <portaudio.h>
#include <pocketsphinx.h>
#include <signal.h>

```

Classes

- class [TTTController](#)

The [TTTController](#) class acts as the controller (C in MVC) for a Tic-Tac-Toe game.

7.24.1 Detailed Description

Definition of the [TTTController](#) class for the Tic-Tac-Toe game controller (MVC pattern)

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.25 TTTController.h

[Go to the documentation of this file.](#)

```

00001 #ifndef TTTCONTROLLER_H
00002 #define TTTCONTROLLER_H
00003 #include "MiniMaxAgent.h"
00004 #include "HumanPlayer.h"
00005 #include "Board.h"
00006 #include "TTTCommonTypes.h"
00007 #include "TicTacToeGame.h"
00008 #include "NetworkManager.h"
00009 #include <QTcpSocket>
00010 #include <QObject>
00011 #include <memory>
00012 #include <vector>
00013 #include <portaudio.h>
00014 #include <pocketsphinx.h>
00015 #include <signal.h>
00016
00017 using std::unique_ptr;
00018 using std::vector;
00019
00061 class TTTController : public QObject
00062 {
00063     Q_OBJECT
00064 protected:
00068     TicTacToeGame view_;
00072     Board board_;
00076     vector<Cell> cells_;
00080     const TTTOptions& options_;
00084     BoardMarks currentPlayer_;
00088     unique_ptr<Player> playerX_;
00092     unique_ptr<Player> playerO_;
00096     NetworkManager* network_;
00097
00098 private: // Methods
00105     void setConnections();
00112     void reset();
00113
00114 protected: // Methods
00123     void updateGameState(Cell &cell);
00127     void switchPlayer();
00128
00129 public:
00139     explicit TTTController(const TTTOptions &options, QObject *parent = nullptr);
00140
00147     virtual void startGame();
00148
00153     QWidget* getView() { return &view_; }
00154
00155 signals:
00156     // --- NET ---
00161     void sendMessage(const QString& msg);
00162
00163     // --- Players ---
00170     void playTurn();
00171
00172 public slots:
00181     void updateGame(Cell &cell);
00182
00183 private slots:
00192     void onNetMessageReceived(const QString& msg);
00193 };
00194
00195 #endif // TTTCONTROLLER_H

```

7.26 include/Model/Board.h File Reference

Definition of the [Board](#) class representing the game model.

```

#include "TTTCommonTypes.h"
#include <QtGlobal>
#include <vector>

```

Classes

- class [Board](#)

The [Board](#) class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

7.26.1 Detailed Description

Definition of the [Board](#) class representing the game model.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.27 Board.h

[Go to the documentation of this file.](#)

```
00001 #ifndef BOARD_H
00002 #define BOARD_H
00003 #include "TTCommonTypes.h"
00004 #include <QtGlobal>
00005 #include <vector>
00006
00007 using std::vector;
00008
00044 class Board
00045 {
00046 protected: // Data
00051     vector<vector<BoardMarks>> board_;
00056     size_t boardSize_;
00057
00058 public:
00063     Board(size_t size);
00068     Board(Board& b);
00073     Board(const Board& b);
00078     Board(Board&& b);
00085     Board& operator=(Board b);
00089     virtual ~Board();
00095     virtual BoardState evaluateBoard() const;
00106     virtual bool setPlayerInput(size_t row, size_t col, BoardMarks currentPlayer);
00114     virtual BoardMarks at(size_t row, size_t col) const;
00118     virtual void reset();
00123     virtual void resetCell(size_t row, size_t col);
00128     size_t size() const;
00129 #ifdef QT_DEBUG
00130     virtual void printBoard () const;
00131 #endif
00132 };
00133
00134 #endif // BOARD_H
```

7.28 include/Network/NetworkManager.h File Reference

Definition of the [NetworkManager](#) class for TCP communication.

```
#include <QObject>
#include <QTcpSocket>
#include <QTimer>
```

Classes

- class [NetworkManager](#)
TCP communication manager class.

7.28.1 Detailed Description

Definition of the [NetworkManager](#) class for TCP communication.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.29 NetworkManager.h

[Go to the documentation of this file.](#)

```
00001 #ifndef NETWORKMANAGER_H
00002 #define NETWORKMANAGER_H
00003 #pragma once
00004 #include <QObject>
00005 #include <QTcpSocket>
00006 #include <QTimer>
00007
00038 class NetworkManager : public QObject
00039 {
00040     Q_OBJECT
00041
00042 public:
00050     explicit NetworkManager(QObject *parent = nullptr);
00051
00061     void connectToServer(const QString &ip, quint16 port);
00062
00070     void sendMessage(const QString &message);
00071
00072 signals:
00076     void connected();
00077
00081     void disconnected();
00082
00087     void errorOccurred(const QString &error);
00088
00093     void messageSent(const QString &message);
```

```

00094
00099     void messageReceived(const QString &message);
00100
00101 private slots:
00107     void onConnected();
00108
00114     void onDisconnected();
00115
00123     void onError(QAbstractSocket::SocketError socketError);
00124
00130     void onSocketReadyRead();
00131
00132 private:
00133     QTcpSocket *socket_;
00134 };
00135
00136 #endif //NETWORKMANAGER_H

```

7.30 include/Speech/SpeechManager.h File Reference

Definition of the [SpeechManager](#) class for voice recognition using Pocketsphinx and PortAudio.

```

#include <QObject>
#include <QString>
#include <QDebug>
#include <QtConcurrent/QtConcurrentRun>
#include <QFuture>
#include <cstdlib>
#include <portaudio.h>
#include <pocketsphinx.h>
#include <signal.h>
#include <ConfigManager.h>

```

Classes

- class [SpeechManager](#)
Voice recognition manager class using Pocketsphinx and PortAudio.

7.30.1 Detailed Description

Definition of the [SpeechManager](#) class for voice recognition using Pocketsphinx and PortAudio.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.31 SpeechManager.h

[Go to the documentation of this file.](#)

```

00001 #ifndef SPEECHMANAGER_H
00002 #define SPEECHMANAGER_H
00003 #include <QObject>
00004 #include <QString>
00005 #include <QDebug>
00006 #include <QtConcurrent/QtConcurrentRun>
00007 #include <QFuture>
00008 #include <cstdint>
00009 #include <portaudio.h>
00010 #include <pocketsphinx.h>
00011 #include <signal.h>
00012 #include <ConfigManager.h>
00013
00050 class SpeechManager : public QObject
00051 {
00052     Q_OBJECT
00053 public:
00062     explicit SpeechManager(QObject *parent = nullptr);
00063
00070     ~SpeechManager();
00071
00079     void configure();
00080
00087     void init();
00088
00095     void startListening();
00096
00097 signals:
00102     void speechRecognized(const QString &text);
00103
00112     void speechReadyToProcess(const char *hyp);
00113
00114 private slots:
00123     void processRecognitionResult(const char *hyp);
00124
00125 private:
00126     PaStream *stream_;
00127     PaError err_;
00128     ps_decoder_t *decoder_;
00129     ps_config_t *config_;
00130     ps_endpointinter_t *ep_;
00131     short *frame_;
00132     size_t frame_size_;
00133     const char *hyp_;
00134
00142     bool hypIsValid();
00143
00150     void freeHeap();
00151 };
00152 #endif // SPEECHMANAGER_H

```

7.32 include/TTTCommonTypes.h File Reference

Definition of common types and constants for the Tic-Tac-Toe game.

```

#include <cstdint>
#include <portaudio.h>
#include <signal.h>

```

Classes

- struct [Cell](#)

The [Cell](#) struct represents a board cell and all the necessary information needed about the cell.

- struct [TTTOptions](#)

The [TTTOptions](#) struct defines a tic tac toe game options.

Namespaces

- namespace [defaults](#)

Enumerations

- enum class [BoardMarks](#) { [Empty](#) , [X](#) , [O](#) }
The BoardMarks enum defines the type of mark on the board.
- enum class [BoardState](#) { [NoWinner](#) , [XWins](#) , [OWins](#) , [Tie](#) }
The BoardState enum defines the possible states of a board.

Variables

- constexpr short [defaults::INVALID_CELL](#) = -1
- constexpr size_t [defaults::DEFAULT_BOARD_SIZE](#) = 3
- constexpr unsigned short [defaults::DEFAULT_MINIMAX_DEPTH](#) = 3
- constexpr unsigned short [defaults::GUI_CELL_ROW_SPAN](#) = 1
- constexpr unsigned short [defaults::GUI_CELL_COLUMN_SPAN](#) = 1
- constexpr char [defaults::X_COLOR](#) [] = "#455A64"
- constexpr char [defaults::O_COLOR](#) [] = "#FF5722"
- constexpr char [defaults::DEFAULT_COLOR](#) [] = "#FFF"

7.32.1 Detailed Description

Definition of common types and constants for the Tic-Tac-Toe game.

Author

Miguel Fernández Lorenzo

Version

1.0

Date

November 2025

7.32.2 Enumeration Type Documentation

7.32.2.1 BoardMarks

```
enum class BoardMarks [strong]
```

The BoardMarks enum defines the type of mark on the board.

Enumerator

Empty	
X	
O	

7.32.2.2 BoardState

```
enum class BoardState [strong]
```

The BoardState enum defines the possible states of a board.

Enumerator

NoWinner	
XWins	
OWins	
Tie	

7.33 TTTCommonTypes.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TTTCOMMONTYPES_H
00002 #define TTTCOMMONTYPES_H
00003 #include <cstdlib>
00004 #include <portaudio.h>
00005 #include <signal.h>
00006
00007 class QPushButton;
00008 namespace defaults
00009 {
00010     constexpr short INVALID_CELL = -1;
00011     constexpr size_t DEFAULT_BOARD_SIZE = 3;
00012     constexpr unsigned short DEFAULT_MINIMAX_DEPTH = 3;
00013     constexpr unsigned short GUI_CELL_ROW_SPAN = 1;
00014     constexpr unsigned short GUI_CELL_COLUMN_SPAN = 1;
00015     constexpr char X_COLOR[] = "#455A64";
00016     constexpr char O_COLOR[] = "#FF5722";
00017     constexpr char DEFAULT_COLOR[] = "#FFF";
00018 }
00019 struct Cell
00020 {
00021     QPushButton *cellBtn = nullptr;
00022     int row = defaults::INVALID_CELL;
00023     int col = defaults::INVALID_CELL;
00024     Cell(QPushButton *cellBtn, int row, int col)
00025         : cellBtn(cellBtn), row(row), col(col) {}
00026 };
00027
00028 struct TTTOptions
00029 {
00030     size_t boardSize = defaults::DEFAULT_BOARD_SIZE;
00031     unsigned short miniMaxDepth = defaults::DEFAULT_MINIMAX_DEPTH;
00032     bool AIopponent = true;
00033     bool AIstarts = false;
00034 };
00035
00036 enum class BoardMarks { Empty, X, O };
00037
00038 enum class BoardState { NoWinner, XWins, OWins, Tie };
00039 #endif // TTTCOMMONTYPES_H
```

7.34 include/View/TicTacToeGame.h File Reference

Definition of the [TicTacToeGame](#) class for the game GUI.

```
#include "TTTCommonTypes.h"
#include "ui_TicTacToeGame.h"
#include "waitingspinnerwidget.h"
```

Classes

- class [TicTacToeGame](#)

The [TicTacToeGame](#) class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

7.34.1 Detailed Description

Definition of the [TicTacToeGame](#) class for the game GUI.

Author

Abdelrahman Osama

Version

1.0

7.35 TicTacToeGame.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MAINWINDOW_H
00002 #define MAINWINDOW_H
00003
00004 #include "TTTCommonTypes.h"
00005 #include "ui_TicTacToeGame.h"
00006 #include "waitingspinnerwidget.h"
00007 #include <QDialog>
00008 #include <vector>
00009 #include <memory>
00010
00011 using std::vector;
00012
00046 class TicTacToeGame final : public QDialog
00047 {
00048     Q_OBJECT
00049
00050 public:
00055     explicit TicTacToeGame(QWidget *parent = nullptr);
00061     void updateCell(Cell &cell, BoardMarks currentPlayer);
00066     void declareGameState(BoardState boardState);
00073     vector<Cell> buildCellButtons(size_t boardSize);
00079     void reset(vector<Cell> &cells);
00080
00084     void updateConnectionStatus(bool);
00085     void updateRecordingStatus(bool);
00086     void updateWaitingStatus(bool);
00092 private:
00096     std::unique_ptr<Ui::TicTacToeGame> ui;
00101     void setConnections();
00107     QString getPlayerText(BoardMarks currentPlayer);
```

```
00113     QString getPlayerStyleSheet(BoardMarks currentPlayer);
00121     QString getBoardFinalStateText(BoardState boardState);
00122
00123     void setLedStyle(QLabel* label, const QString& text, const QString& bgColor, const QString&
    textColor, int fontSize);
00127     WaitingSpinnerWidget* waiter;
00128
00129
00130 signals:
00135     void newGame();
00139     void goBack();
00140 };
00141
00142 #endif // MAINWINDOW_H
```

7.36 include/View/TitleScreen.h File Reference

Definition of the [TitleScreen](#) class for the main menu GUI.

```
#include "TTTCommonTypes.h"
#include "ui_titlescreen.h"
#include <QMainWindow>
#include <memory>
```

Classes

- class [TitleScreen](#)

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

7.36.1 Detailed Description

Definition of the [TitleScreen](#) class for the main menu GUI.

Author

Abdelrahman Osama

Version

1.0

Date

November 2025

7.37 TitleScreen.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TITLES_SCREEN_H
00002 #define TITLES_SCREEN_H
00003 #include "TTCommonTypes.h"
00004 #include "ui_titlescreen.h"
00005 #include <QMainWindow>
00006 #include <memory>
00007
00037 class TitleScreen final : public QMainWindow
00038 {
00039     Q_OBJECT
00040
00041 public:
00046     explicit TitleScreen(QWidget *parent = nullptr);
00047
00048 private:
00052     std::unique_ptr<Ui::TitleScreen> ui;
00056     void setConnections();
00060     TTOptions options_;
00065     void closeEvent(QCloseEvent *event) override;
00066
00067 public slots:
00072     void updateSinglePlayer(bool checked);
00077     void updateTwoPlayers(bool checked);
00082     void updateAIStartsGame(bool checked);
00087     void updateMiniMaxDepth(int depth);
00091     void startGame();
00092
00093 signals:
00097     void exited();
00098 };
00099
00100 #endif // TITLES_SCREEN_H
```

7.38 include/View/waitingspinnerwidget.h File Reference

```
#include <QWidget>
#include <QTimer>
#include <QColor>
```

Classes

- class [WaitingSpinnerWidget](#)

7.39 waitingspinnerwidget.h

[Go to the documentation of this file.](#)

```
00001 /* Original Work Copyright (c) 2012-2014 Alexander Turkin
00002    Modified 2014 by William Hallatt
00003    Modified 2015 by Jacob Dawid
00004 Permission is hereby granted, free of charge, to any person obtaining a copy of
00005 this software and associated documentation files (the "Software"), to deal in
00006 the Software without restriction, including without limitation the rights to
00007 use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
00008 the Software, and to permit persons to whom the Software is furnished to do so,
00009 subject to the following conditions:
00010
00011 The above copyright notice and this permission notice shall be included in all
00012 copies or substantial portions of the Software.
00013
00014 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
00016 FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR
00017 COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
00018 IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
```

```

00019 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00020 */
00021
00022 #pragma once
00023
00024 // Qt includes
00025 #include <QWidget>
00026 #include <QTimer>
00027 #include <QColor>
00028
00029 «««< HEAD
00030 =====
00031 >>>> afdadd3d46bec0902f02aec6604c627e510e4f67
00032 class WaitingSpinnerWidget : public QWidget {
00033     Q_OBJECT
00034 public:
00035     WaitingSpinnerWidget(QWidget *parent = 0,
00036                          bool centerOnParent = true,
00037                          bool disableParentWhenSpinning = true);
00038
00039     WaitingSpinnerWidget(Qt::WindowModality modality,
00040                          QWidget *parent = 0,
00041                          bool centerOnParent = true,
00042                          bool disableParentWhenSpinning = true);
00043
00044 public slots:
00045     void start();
00046     void stop();
00047
00048 public:
00049     void setColor(QColor color);
00050     void setRoundness(qreal roundness);
00051     void setMinimumTrailOpacity(qreal minimumTrailOpacity);
00052     void setTrailFadePercentage(qreal trail);
00053     void setRevolutionsPerSecond(qreal revolutionsPerSecond);
00054     void setNumberOfLines(int lines);
00055     void setLineLength(int length);
00056     void setLineWidth(int width);
00057     void setInnerRadius(int radius);
00058     void setText(QString text);
00059
00060     QColor color();
00061     qreal roundness();
00062     qreal minimumTrailOpacity();
00063     qreal trailFadePercentage();
00064     qreal revolutionsPersSecond();
00065     int numberOfLines();
00066     int lineLength();
00067     int lineWidth();
00068     int innerRadius();
00069
00070     bool isSpinning() const;
00071
00072 private slots:
00073     void rotate();
00074
00075 protected:
00076     void paintEvent(QPaintEvent *paintEvent);
00077
00078 private:
00079     static int lineCountDistanceFromPrimary(int current, int primary,
00080                                             int totalNrOfLines);
00081     static QColor currentLineColor(int distance, int totalNrOfLines,
00082                                    qreal trailFadePerc, qreal minOpacity,
00083                                    QColor color);
00084
00085     void initialize();
00086     void updateSize();
00087     void updateTimer();
00088     void updatePosition();
00089
00090 private:
00091     QColor _color;
00092     qreal _roundness; // 0..100
00093     qreal _minimumTrailOpacity;
00094     qreal _trailFadePercentage;
00095     qreal _revolutionsPerSecond;
00096     int _numberOfLines;
00097     int _lineLength;
00098     int _lineWidth;
00099     int _innerRadius;
00100
00101 private:
00102     WaitingSpinnerWidget(const WaitingSpinnerWidget&);
00103     WaitingSpinnerWidget& operator=(const WaitingSpinnerWidget&);
00104
00105     QTimer *_timer;

```

```
00135     bool     _centerOnParent;  
00136     bool     _disableParentWhenSpinning;  
00137     int      _currentCounter;  
00138     bool     _isSpinning;  
00139 };
```

7.40 src/AI/HumanPlayer.cpp File Reference

```
#include "HumanPlayer.h"
```

7.41 src/AI/MiniMaxAgent.cpp File Reference

```
#include "MiniMaxAgent.h"  
#include <QPair>  
#include <limits.h>
```

7.42 src/AI/Player.cpp File Reference

```
#include "Player.h"  
#include "moc_Player.cpp"
```

7.43 src/Config/ConfigManager.cpp File Reference

```
#include "ConfigManager.h"
```

7.44 src/Controller/TTTController.cpp File Reference

```
#include "TTTController.h"  
#include "MiniMaxAgent.h"  
#include "TicTacToeGame.h"  
#include <QPushButton>  
#include <QtConcurrent/QtConcurrentRun>  
#include <QFuture>
```

7.45 src/main.cpp File Reference

```
#include "TicTacToeGame.h"  
#include "TitleScreen.h"  
#include "ConfigManager.h"  
#include <QApplication>
```

Functions

- int [main](#) (int argc, char *argv[])

7.45.1 Function Documentation

7.45.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

References [TitleScreen::exited\(\)](#), [ConfigManager::instance\(\)](#), and [ConfigManager::load\(\)](#).

7.46 src/Model/Board.cpp File Reference

```
#include "Board.h"
#include <QString>
#include <stdexcept>
```

7.47 src/Network/NetworkManager.cpp File Reference

```
#include "NetworkManager.h"
#include "TTTController.h"
#include <QDebug>
```

7.48 src/Speech/SpeechManager.cpp File Reference

```
#include "SpeechManager.h"
```

7.49 src/View/TicTacToeGame.cpp File Reference

```
#include "TicTacToeGame.h"
#include <QMessageBox>
#include <QPushButton>
```

7.50 src/View/TitleScreen.cpp File Reference

```
#include "TitleScreen.h"
#include "TTTController.h"
```

7.51 src/waitingspinnerwidget.cpp File Reference

```
#include "waitingspinnerwidget.h"  
#include <cmath>  
#include <algorithm>  
#include <QPainter>  
#include <QTimer>
```


Index

- ~centerOnParent
 - WaitingSpinnerWidget, 70
- ~color
 - WaitingSpinnerWidget, 70
- ~currentCounter
 - WaitingSpinnerWidget, 71
- ~disableParentWhenSpinning
 - WaitingSpinnerWidget, 71
- ~innerRadius
 - WaitingSpinnerWidget, 71
- ~isSpinning
 - WaitingSpinnerWidget, 71
- ~lineLength
 - WaitingSpinnerWidget, 71
- ~lineWidth
 - WaitingSpinnerWidget, 71
- ~minimumTrailOpacity
 - WaitingSpinnerWidget, 71
- ~numberOfLines
 - WaitingSpinnerWidget, 71
- ~revolutionsPerSecond
 - WaitingSpinnerWidget, 72
- ~roundness
 - WaitingSpinnerWidget, 72
- ~timer
 - WaitingSpinnerWidget, 72
- ~trailFadePercentage
 - WaitingSpinnerWidget, 72
- ~AIAgent
 - AIAgent, 12
- ~Board
 - Board, 14
- ~ConfigManager
 - ConfigManager, 20
- ~HumanPlayer
 - HumanPlayer, 26
- ~Player
 - Player, 39
- ~SpeechManager
 - SpeechManager, 42
- AI_WIN_SCORE
 - MiniMaxAgent, 32
- AIAgent, 11
 - ~AIAgent, 12
 - AIAgent, 12
- Alopponent
 - TTTOptions, 63
- Alstarts
 - TTTOptions, 63

- at
 - Board, 15
- baseDir_
 - ConfigManager, 23
- Board, 13
 - ~Board, 14
 - at, 15
 - Board, 14
 - board_, 17
 - boardSize_, 17
 - evaluateBoard, 15
 - operator=, 15
 - printBoard, 16
 - reset, 16
 - resetCell, 16
 - setPlayerInput, 16
 - size, 17
- board_
 - Board, 17
 - Player, 40
 - TTTController, 61
- BoardMarks
 - TTTCommonTypes.h, 85
- boardSize
 - TTTOptions, 63
- boardSize_
 - Board, 17
- BoardState
 - TTTCommonTypes.h, 86
- buildCellButtons
 - TicTacToeGame, 48
- Cell, 18
 - Cell, 18
 - cellBtn, 19
 - col, 19
 - row, 19
- cellBtn
 - Cell, 19
- cells_
 - TTTController, 61
- closeEvent
 - TitleScreen, 54
- col
 - Cell, 19
- color
 - WaitingSpinnerWidget, 66
- config_
 - SpeechManager, 45

- ConfigManager, 19
 - ~ConfigManager, 20
 - baseDir_, 23
 - ConfigManager, 20, 21
 - getBool, 21
 - getInt, 21
 - getValue, 22
 - instance, 22
 - load, 22
 - operator=, 23
 - resolvePath, 23
 - settings_, 24
- configure
 - SpeechManager, 43
- connected
 - NetworkManager, 35
- connectToServer
 - NetworkManager, 35
- currentLineColor
 - WaitingSpinnerWidget, 66
- currentPlayer_
 - TTTController, 61
- declareGameState
 - TicTacToeGame, 48
- decoder_
 - SpeechManager, 45
- DEFAULT_BOARD_SIZE
 - defaults, 9
- DEFAULT_COLOR
 - defaults, 9
- DEFAULT_MINIMAX_DEPTH
 - defaults, 9
- defaults, 9
 - DEFAULT_BOARD_SIZE, 9
 - DEFAULT_COLOR, 9
 - DEFAULT_MINIMAX_DEPTH, 9
 - GUI_CELL_COLUMN_SPAN, 9
 - GUI_CELL_ROW_SPAN, 10
 - INVALID_CELL, 10
 - O_COLOR, 10
 - X_COLOR, 10
- depth_
 - MiniMaxAgent, 32
- disconnected
 - NetworkManager, 35
- docs/plantuml/AIAgent.puml, 73
- docs/plantuml/Board.puml, 73
- docs/plantuml/ConfigManager.puml, 73
- docs/plantuml/HumanPlayer.puml, 73
- docs/plantuml/MiniMaxAgent.puml, 73
- docs/plantuml/NetworkManager.puml, 73
- docs/plantuml/Player.puml, 73
- docs/plantuml/SpeechManager.puml, 73
- docs/plantuml/TicTacToeGame.puml, 73
- docs/plantuml/TitleScreen.puml, 73
- docs/plantuml/TTTCommonTypes.puml, 73
- docs/plantuml/TTTController.puml, 73
- docs/plantuml/WaitingSpinnerWidget.puml, 73
- Empty
 - TTTCommonTypes.h, 86
- ep_
 - SpeechManager, 45
- err_
 - SpeechManager, 45
- errorOccurred
 - NetworkManager, 35
- evaluateBoard
 - Board, 15
- exited
 - TitleScreen, 54
- frame_
 - SpeechManager, 46
- frame_size_
 - SpeechManager, 46
- freeHeap
 - SpeechManager, 43
- getBoardFinalStateText
 - TicTacToeGame, 49
- getBool
 - ConfigManager, 21
- getInt
 - ConfigManager, 21
- getPlayerStyleSheet
 - TicTacToeGame, 49
- getPlayerText
 - TicTacToeGame, 50
- getValue
 - ConfigManager, 22
- getView
 - TTTController, 58
- goBack
 - TicTacToeGame, 50
- GUI_CELL_COLUMN_SPAN
 - defaults, 9
- GUI_CELL_ROW_SPAN
 - defaults, 10
- handleThreadCleanup
 - HumanPlayer, 26
- HumanPlayer, 24
 - ~HumanPlayer, 26
 - handleThreadCleanup, 26
 - HumanPlayer, 25
 - parseSpeechCommand, 26
 - play, 26
 - startListening, 27
 - stopListening, 27
 - stt_, 28
 - sttThread_, 28
 - validateCommand, 27
- hyp_
 - SpeechManager, 46
- hypIsValid
 - SpeechManager, 43

- include/AI/AIAgent.h, [73](#), [74](#)
- include/AI/HumanPlayer.h, [74](#), [75](#)
- include/AI/MiniMaxAgent.h, [75](#), [76](#)
- include/AI/Player.h, [77](#)
- include/Config/ConfigManager.h, [78](#)
- include/Controller/TTTController.h, [79](#), [80](#)
- include/Model/Board.h, [80](#), [81](#)
- include/Network/NetworkManager.h, [82](#)
- include/Speech/SpeechManager.h, [83](#), [84](#)
- include/TTTCommonTypes.h, [84](#), [86](#)
- include/View/TicTacToeGame.h, [87](#)
- include/View/TitleScreen.h, [88](#), [89](#)
- include/View/waitingspinnerwidget.h, [89](#)
- init
 - SpeechManager, [43](#)
- initialize
 - WaitingSpinnerWidget, [66](#)
- innerRadius
 - WaitingSpinnerWidget, [66](#)
- instance
 - ConfigManager, [22](#)
- INVALID_CELL
 - defaults, [10](#)
- isSpinning
 - WaitingSpinnerWidget, [66](#)
- lineCountDistanceFromPrimary
 - WaitingSpinnerWidget, [66](#)
- lineLength
 - WaitingSpinnerWidget, [67](#)
- lineWidth
 - WaitingSpinnerWidget, [67](#)
- load
 - ConfigManager, [22](#)
- main
 - main.cpp, [92](#)
- main.cpp
 - main, [92](#)
- mark_
 - Player, [40](#)
- maxMove
 - MiniMaxAgent, [30](#)
- messageReceived
 - NetworkManager, [36](#)
- messageSent
 - NetworkManager, [36](#)
- MiniMaxAgent, [28](#)
 - AI_WIN_SCORE, [32](#)
 - depth_, [32](#)
 - maxMove, [30](#)
 - MiniMaxAgent, [30](#)
 - minMove, [31](#)
 - play, [31](#)
 - PLAYER_WIN_SCORE, [33](#)
 - playerMark_, [31](#)
 - score, [32](#)
 - TIE_SCORE, [33](#)
- miniMaxDepth
 - TTTOptions, [63](#)
- minimumTrailOpacity
 - WaitingSpinnerWidget, [67](#)
- minMove
 - MiniMaxAgent, [31](#)
- network_
 - TTTController, [61](#)
- NetworkManager, [33](#)
 - connected, [35](#)
 - connectToServer, [35](#)
 - disconnected, [35](#)
 - errorOccurred, [35](#)
 - messageReceived, [36](#)
 - messageSent, [36](#)
 - NetworkManager, [34](#)
 - onConnected, [36](#)
 - onDisconnected, [36](#)
 - onError, [37](#)
 - onSocketReadyRead, [37](#)
 - sendMessage, [37](#)
 - socket_, [38](#)
- newGame
 - TicTacToeGame, [50](#)
- NoWinner
 - TTTCommonTypes.h, [86](#)
- numberOfLines
 - WaitingSpinnerWidget, [67](#)
- O
 - TTTCommonTypes.h, [86](#)
- O_COLOR
 - defaults, [10](#)
- onConnected
 - NetworkManager, [36](#)
- onDisconnected
 - NetworkManager, [36](#)
- onError
 - NetworkManager, [37](#)
- onNetworkMessageReceived
 - TTTController, [59](#)
- onSocketReadyRead
 - NetworkManager, [37](#)
- operator=
 - Board, [15](#)
 - ConfigManager, [23](#)
 - WaitingSpinnerWidget, [67](#)
- options_
 - TitleScreen, [56](#)
 - TTTController, [62](#)
- OWins
 - TTTCommonTypes.h, [86](#)
- paintEvent
 - WaitingSpinnerWidget, [67](#)
- parseSpeechCommand
 - HumanPlayer, [26](#)
- play
 - HumanPlayer, [26](#)

- MiniMaxAgent, 31
- Player, 39
- Player, 38
 - ~Player, 39
 - board_, 40
 - mark_, 40
 - play, 39
 - Player, 39
 - playerFinished, 40
- PLAYER_WIN_SCORE
 - MiniMaxAgent, 33
- playerFinished
 - Player, 40
- playerMark_
 - MiniMaxAgent, 31
- playerO_
 - TTTController, 62
- playerX_
 - TTTController, 62
- playTurn
 - TTTController, 59
- printBoard
 - Board, 16
- processRecognitionResult
 - SpeechManager, 44
- reset
 - Board, 16
 - TicTacToeGame, 50
 - TTTController, 59
- resetCell
 - Board, 16
- resolvePath
 - ConfigManager, 23
- revolutionsPersSecond
 - WaitingSpinnerWidget, 67
- rotate
 - WaitingSpinnerWidget, 68
- roundness
 - WaitingSpinnerWidget, 68
- row
 - Cell, 19
- score
 - MiniMaxAgent, 32
- sendMessage
 - NetworkManager, 37
 - TTTController, 59
- setColor
 - WaitingSpinnerWidget, 68
- setConnections
 - TicTacToeGame, 51
 - TitleScreen, 54
 - TTTController, 60
- setInnerRadius
 - WaitingSpinnerWidget, 68
- setLedStyle
 - TicTacToeGame, 51
- setLineLength
 - WaitingSpinnerWidget, 68
- setLineWidth
 - WaitingSpinnerWidget, 68
- setMinimumTrailOpacity
 - WaitingSpinnerWidget, 68
- setNumberOfLines
 - WaitingSpinnerWidget, 69
- setPlayerInput
 - Board, 16
- setRevolutionsPerSecond
 - WaitingSpinnerWidget, 69
- setRoundness
 - WaitingSpinnerWidget, 69
- setText
 - WaitingSpinnerWidget, 69
- settings_
 - ConfigManager, 24
- setTrailFadePercentage
 - WaitingSpinnerWidget, 69
- size
 - Board, 17
- socket_
 - NetworkManager, 38
- SpeechManager, 41
 - ~SpeechManager, 42
 - config_, 45
 - configure, 43
 - decoder_, 45
 - ep_, 45
 - err_, 45
 - frame_, 46
 - frame_size_, 46
 - freeHeap, 43
 - hyp_, 46
 - hyplsValid, 43
 - init, 43
 - processRecognitionResult, 44
 - SpeechManager, 42
 - speechReadyToProcess, 44
 - speechRecognized, 44
 - startListening, 45
 - stream_, 46
- speechReadyToProcess
 - SpeechManager, 44
- speechRecognized
 - SpeechManager, 44
- src/Al/HumanPlayer.cpp, 91
- src/Al/MiniMaxAgent.cpp, 91
- src/Al/Player.cpp, 91
- src/Config/ConfigManager.cpp, 91
- src/Controller/TTTController.cpp, 91
- src/main.cpp, 91
- src/Model/Board.cpp, 92
- src/Network/NetworkManager.cpp, 92
- src/Speech/SpeechManager.cpp, 92
- src/View/TicTacToeGame.cpp, 92
- src/View/TitleScreen.cpp, 92
- src/waitingspinnerwidget.cpp, 93

- start
 - WaitingSpinnerWidget, 69
- startGame
 - TitleScreen, 55
 - TTTController, 60
- startListening
 - HumanPlayer, 27
 - SpeechManager, 45
- stop
 - WaitingSpinnerWidget, 69
- stopListening
 - HumanPlayer, 27
- stream_
 - SpeechManager, 46
- stt_
 - HumanPlayer, 28
- sttThread_
 - HumanPlayer, 28
- switchPlayer
 - TTTController, 60
- TicTacToeGame, 47
 - buildCellButtons, 48
 - declareGameState, 48
 - getBoardFinalStateText, 49
 - getPlayerStyleSheet, 49
 - getPlayerText, 50
 - goBack, 50
 - newGame, 50
 - reset, 50
 - setConnections, 51
 - setLedStyle, 51
 - TicTacToeGame, 48
 - ui, 52
 - updateCell, 51
 - updateConnectionStatus, 51
 - updateRecordingStatus, 52
 - updateWaitingStatus, 52
 - waiter, 52
- Tie
 - TTTCommonTypes.h, 86
- TIE_SCORE
 - MiniMaxAgent, 33
- TitleScreen, 53
 - closeEvent, 54
 - exited, 54
 - options_, 56
 - setConnections, 54
 - startGame, 55
 - TitleScreen, 54
 - ui, 56
 - updateAlStartsGame, 55
 - updateMiniMaxDepth, 55
 - updateSinglePlayer, 55
 - updateTwoPlayers, 56
- trailFadePercentage
 - WaitingSpinnerWidget, 70
- TTTCommonTypes.h
 - BoardMarks, 85
 - BoardState, 86
 - Empty, 86
 - NoWinner, 86
 - O, 86
 - OWins, 86
 - Tie, 86
 - X, 86
 - XWins, 86
- TTTController, 57
 - board_, 61
 - cells_, 61
 - currentPlayer_, 61
 - getView, 58
 - network_, 61
 - onNetworkMessageReceived, 59
 - options_, 62
 - playerO_, 62
 - playerX_, 62
 - playTurn, 59
 - reset, 59
 - sendMessage, 59
 - setConnections, 60
 - startGame, 60
 - switchPlayer, 60
 - TTTController, 58
 - updateGame, 60
 - updateGameState, 61
 - view_, 62
- TTTOptions, 62
 - Alopponent, 63
 - AlStarts, 63
 - boardSize, 63
 - miniMaxDepth, 63
- ui
 - TicTacToeGame, 52
 - TitleScreen, 56
- updateAlStartsGame
 - TitleScreen, 55
- updateCell
 - TicTacToeGame, 51
- updateConnectionStatus
 - TicTacToeGame, 51
- updateGame
 - TTTController, 60
- updateGameState
 - TTTController, 61
- updateMiniMaxDepth
 - TitleScreen, 55
- updatePosition
 - WaitingSpinnerWidget, 70
- updateRecordingStatus
 - TicTacToeGame, 52
- updateSinglePlayer
 - TitleScreen, 55
- updateSize
 - WaitingSpinnerWidget, 70
- updateTimer
 - WaitingSpinnerWidget, 70

- updateTwoPlayers
 - TitleScreen, [56](#)
- updateWaitingStatus
 - TicTacToeGame, [52](#)
- validateCommand
 - HumanPlayer, [27](#)
- view_
 - TTTController, [62](#)
- waiter
 - TicTacToeGame, [52](#)
- WaitingSpinnerWidget, [63](#)
 - _centerOnParent, [70](#)
 - _color, [70](#)
 - _currentCounter, [71](#)
 - _disableParentWhenSpinning, [71](#)
 - _innerRadius, [71](#)
 - _isSpinning, [71](#)
 - _lineLength, [71](#)
 - _lineWidth, [71](#)
 - _minimumTrailOpacity, [71](#)
 - _numberOfLines, [71](#)
 - _revolutionsPerSecond, [72](#)
 - _roundness, [72](#)
 - _timer, [72](#)
 - _trailFadePercentage, [72](#)
 - color, [66](#)
 - currentLineColor, [66](#)
 - initialize, [66](#)
 - innerRadius, [66](#)
 - isSpinning, [66](#)
 - lineCountDistanceFromPrimary, [66](#)
 - lineLength, [67](#)
 - lineWidth, [67](#)
 - minimumTrailOpacity, [67](#)
 - numberOfLines, [67](#)
 - operator=, [67](#)
 - paintEvent, [67](#)
 - revolutionsPersSecond, [67](#)
 - rotate, [68](#)
 - roundness, [68](#)
 - setColor, [68](#)
 - setInnerRadius, [68](#)
 - setLineLength, [68](#)
 - setLineWidth, [68](#)
 - setMinimumTrailOpacity, [68](#)
 - setNumberOfLines, [69](#)
 - setRevolutionsPerSecond, [69](#)
 - setRoundness, [69](#)
 - setText, [69](#)
 - setTrailFadePercentage, [69](#)
 - start, [69](#)
 - stop, [69](#)
 - trailFadePercentage, [70](#)
 - updatePosition, [70](#)
 - updateSize, [70](#)
 - updateTimer, [70](#)
 - WaitingSpinnerWidget, [65](#)
- X
 - TTTCommonTypes.h, [86](#)
- X_COLOR
 - defaults, [10](#)
- XWins
 - TTTCommonTypes.h, [86](#)